

Guía Integral de Estudio: Ingeniería y Calidad de Software (Filminas 01 a 08)

Fuente: Filminas de Cátedra 01 a 08 | Apuntes Teóricos de Clase | Resumen Teórico Final | Guía SCRUM 2020 y Expansión 2026

Índice General de Contenidos

Apartado Especial: Paradigma Ágil vs. Paradigma Tradicional

El Triángulo de Hierro Invertido: Restricción Triple Tradicional vs. Triángulo Ágil
Gestión Predictiva (por Plan) vs. Gestión Adaptativa (por Valor)
Estructura Organizacional: Silos Jerárquicos vs. Equipos Multifuncionales y Autogestionados
Evolución y Mitigación del Riesgo: Big Bang vs. Entrega Continua de Incrementos
Matriz Comparativa Exhaustiva: Paradigma Tradicional vs. Paradigma Ágil

Unidad 1: Fundamentos de la Ingeniería de Software y Gestión de Procesos

Introducción a la Ingeniería del Software: Definición y Diferencia con Software
Naturaleza del Software vs. Manufactura Tradicional
La Crisis del Software y sus Causas Fundamentales
El Paper "No Silver Bullet": Dificultades Esenciales vs. Accidentales
Disciplinas de la Ingeniería de Software y Evolución del Enfoque Sistémico
Procesos de Desarrollo de Software: Definidos vs. Empíricos
Cuadro Comparativo Exhaustivo: Procesos Definidos vs. Procesos Empíricos
Los Tres Pilares del Empirismo en el Software
Modelos de Proceso y Ciclos de Vida del Software
Análisis Comparativo de Modelos de Ciclos de Vida
Criterios de Selección del Modelo de Ciclo de Vida
Componentes de un Proyecto de Desarrollo de Software y las 4 P's
Leyes Fundamentales de Fred Brooks y Dinámica de Proyectos
Vínculo Sistémico: Proceso - Proyecto - Producto
Proyectos Exitosos vs. Proyectos Fallidos: Métricas y Lecciones

Unidad 2: Gestión de Producto, Requerimientos Ágiles, Estimaciones y Scrum

Módulo 2.1: Gestión Estratégica de Producto de Software

Filosofía de Producto vs. Proyecto: Del Enfoque Temporal a la Solución Viva
El Enfoque Lean Startup: Ciclo Crear - Medir - Aprender y Validación de Hipótesis
El Espectro de los Productos Mínimos: MVP vs. MMP vs. MLP
Dual-Track Agile: Product Discovery vs. Product Delivery
Design Thinking y el Esquema del Doble Diamante
Visión del Producto y Plantilla de Elevator Pitch
Product Roadmaps: De Roadmaps de Características a Roadmaps Basados en Resultados

Módulo 2.2: Requerimientos Ágiles y User Stories

Fundamentos de Requerimientos Ágiles: El Manifiesto Ágil y sus Principios
Requerimientos en Ágil y la Superación del BRUF (Big Requirements Up Front)
Jerarquía de Abstracción: Temas, Épicas e Historias de Usuario
Anatomía de una User Story y el Modelo de las 3 C's
Criterios de Aceptación y Pruebas con Formato BDD
Criterios de Calidad INVEST para Historias de Usuario
Técnicas de División Vertical de Historias (Story Slicing)

Spikes de Investigación: Técnicos y Funcionales

User Story Mapping: Estructuración Visual del Backlog

Usuarios Representantes (Proxies) y Gestión de Stakeholders

Módulo 2.3: Estimaciones en Ambientes Ágiles

Naturaleza de la Estimación en Software: Incertidumbre, Probabilidad y Cono de Incertidumbre

Matriz de Desambiguación: Objetivo vs. Estimación vs. Compromiso

Estimaciones Relativas vs. Absolutas: Psicología y Story Points

Escalas de Estimación y la Serie de Fibonacci Modificada

Dinámica del Planning Poker (Scrum Poker) Paso a Paso

Velocidad del Equipo, Capacidad y la Ecuación Central Ágil

Los Cuatro Fundamentos que Hacen Ágiles a las Estimaciones

Módulo 2.4: Framework Scrum (Guía 2020 y Expansión 2026)

Fundamentos de Scrum: Definición, Pilares Empíricos y Valores

Estructura del Scrum Team y Roles / Accountabilities

La Expansión Scrum 2026: SGEP y Nuevas Dinámicas de Trabajo

Los Eventos de Scrum: Entradas, Time-Box y Definición Completa

Los Tres Artefactos Oficiales y sus Compromisos

Planificación de Release y Métricas Ágiles en Scrum

Unidad 3: Software Configuration Management (Gestión de Configuración de Software)

Definición y Alcance de SCM: La Disciplina Paraguas

Conceptos Nucleares: Ítem de Configuración (IC), Versiones, Revisiones y Variantes

Líneas Base (Baselines) y Momentos Críticos del Ciclo de Vida

Las Cuatro Actividades Fundamentales de SCM

El Comité de Control de Cambios (CCB) y el Flujo Formal de Solicitudes de Cambio (CR)

Auditorías de Configuración: FCA (Funcional) y PCA (Física)

Trazabilidad Bidireccional Pre-ERS y Post-ERS

Gestión de Repositorios, Permisos y Buenas Prácticas de SCM

SCM en Entornos Ágiles y Cultura DevOps

Apartado Especial: Paradigma Ágil vs. Paradigma Tradicional

Fuente: filmas de clase (01, 03, 05, 06) | apunte de clase: Requerimientos Ágiles y Componentes de Proyecto | resumen final: páginas (36-41)

La evolución de la ingeniería de software ha estado marcada por la tensión constante entre dos filosofías de trabajo diametralmente opuestas: el enfoque tradicional (predictivo, orientado a

planes y procesos definidos) y el enfoque ágil (adaptativo, orientado al valor y basado en el empirismo). Comprender esta distinción no es una cuestión meramente terminológica, sino un requisito conceptual indispensable para abordar con éxito las decisiones de arquitectura, gestión de proyectos y diseño de procesos en entornos profesionales y evaluaciones académicas.

El Triángulo de Hierro Invertido: Restricción Triple Tradicional vs. Triángulo Ágil

Fuente: filminas de clase (06, diapositivas 18-20) | apunte de clase: Componentes de Proyecto de Desarrollo de SW | resumen final: páginas (36-41)

El concepto clásico de la gestión de proyectos establece que todo desarrollo está gobernado por una **Restricción Triple** interdependiente: **Alcance (Scope)**, **Tiempo (Schedule)** y **Costo (Cost/Resources)**, donde la **Calidad** actúa como el núcleo resultante de equilibrar adecuadamente estas tres dimensiones.

Enfoque Tradicional (Plan-Driven)

Alcance Fijo: Se asume que todos los requerimientos pueden y deben definirse al 100% al inicio del proyecto (ERS).

Tiempo y Costo Estimados (Variables): Se calculan y derivan de forma determinista en función del alcance exhaustivo.

Consecuencia ante Desvíos: Si se producen retrasos o pedidos de cambio, se sobrepasan los plazos y presupuestos, o se termina sacrificando la calidad de forma encubierta (recorte de testing y diseño).

Enfoque Ágil (Value-Driven)

Tiempo y Costo Fijos: Se congelan los plazos mediante iteraciones de longitud fija (timeboxes de 2 a 4 semanas) y se mantiene un equipo multidisciplinario estable.

Alcance Estimado (Variable): El alcance se descompone en ítems priorizados por valor de negocio y se ajusta continuamente.

Consecuencia ante Desvíos: La calidad se mantiene no negociable. Si el tiempo apremia, se renegocia el alcance postergando historias de menor valor, garantizando siempre software funcional en producción.

REGLA DE EXAMEN: LA CALIDAD COMO VARIABLE NO NEGOCIABLE

En la gestión tradicional, cuando el alcance es fijo y el cliente exige mantener la fecha de entrega y el presupuesto sin modificaciones, el equipo cae inevitablemente en la trampa de recortar actividades esenciales de ingeniería (análisis riguroso, pruebas unitarias y de integración). El agilismo resuelve este dilema invirtiendo el triángulo: al fijar el tiempo y el costo mediante ciclos cortos y predecibles, la única variable de ajuste es el alcance secundario. La calidad se convierte en una constante inamovible gracias a la automatización de pruebas y la Definición de Hecho (Definition of Done).

Gestión Predictiva (por Plan) vs. Gestión Adaptativa (por Valor)

Fuente: filmas de clase (01, 03) | apunte de clase: Requerimientos Ágiles y User Stories | resumen final: páginas (36-39)

La diferencia paradigmática fundamental entre ambos modelos radica en cómo conciben la certidumbre y el cambio a lo largo del ciclo de vida del software:

1. Gestión Predictiva (Orientada al Plan):

- **Premisa Teórica:** Concibe el desarrollo de software como un proceso que puede planificarse con exactitud matemática desde la fase inicial. Asume que el cliente sabe con certeza qué necesita y que los requerimientos se mantendrán estables.
- **Actitud ante el Cambio:** El cambio es interpretado como una falla de la fase de análisis previa o como una amenaza al cronograma. Por ende, se establecen barreras burocráticas estrictas (solicitudes formales de cambio, comités y renegociaciones de contratos) para desalentarlo y proteger el plan inicial.
- **Métrica de Éxito:** El proyecto es considerado exitoso si se ajusta rigurosamente al cronograma y al presupuesto aprobados ("on time, on budget"), independientemente de si la solución construida sigue siendo relevante para el negocio en el momento del despliegue.

2. Gestión Adaptativa (Orientada al Valor):

- **Premisa Teórica:** Reconoce que el software se construye en entornos de alta incertidumbre y complejidad. Asume como axioma que al menos el 50% de los requerimientos son emergentes y solo se descubrirán cuando los usuarios interactúen con software real en funcionamiento.
- **Actitud ante el Cambio:** El cambio no se combate; se le da la bienvenida como una oportunidad para generar una ventaja competitiva. El proceso está estructurado para recibir modificaciones de requerimientos incluso en etapas avanzadas de la construcción.
- **Métrica de Éxito:** La medida primordial del progreso es la entrega frecuente y temprana de software funcionando que genera un resultado de negocio tangible (Outcome) en lugar de un mero volumen de código entregado (Output).

Estructura Organizacional: Silos Jerárquicos vs. Equipos Multifuncionales y Autogestionados

Fuente: filmas de clase (03, 06) | apunte de clase: Componentes de Proyecto de Desarrollo de SW | resumen final: páginas (16-18)

La distribución del trabajo y la conformación de los equipos reflejan de forma nítida la divergencia estructural entre ambos mundos:

- **Silos Funcionales y Estructuras Jerárquicas (Tradicional):**

- La organización se divide por especialidades técnicas: Departamento de Análisis Funcional, Equipo de Arquitectura, Grupo de Programación, Departamento de Calidad/Testing y Operaciones/Despliegue.
- La interacción entre etapas se produce mediante "pasajes de testimonio" documentales (*hand-offs*): los analistas completan la ERS y la entregan a los diseñadores; estos producen diagramas y los transfieren a los programadores; finalmente, el código compilado se pasa a testing.
- Este modelo genera pérdidas críticas de contexto, distorsiones cognitivas, deslindamiento de responsabilidades y tiempos muertos masivos entre disciplinas. La toma de decisiones está concentrada en figuras de autoridad formal (Jefe de Proyecto o Gerente de Área).

- **Equipos Multifuncionales y Autogestionados (Ágil):**

- Los equipos son células pequeñas (habitualmente de 3 a 9 profesionales) que contienen **todas las competencias necesarias** para transformar una necesidad de negocio en un incremento funcional desplegado en producción: análisis, desarrollo, pruebas, arquitectura y despliegue conviven en la misma unidad de trabajo.
- No existen jerarquías internas ni subequipos de testing o programación. La responsabilidad por la calidad y el cumplimiento del objetivo del sprint recae de manera colectiva sobre todo el equipo.
- La autogestión (*self-management*) faculta a los profesionales técnicos a decidir internamente **cómo** llevar a cabo el trabajo, eliminando la microgestión y permitiendo que las decisiones técnicas emerjan de quienes están más cerca del código y del problema.

Evolución y Mitigación del Riesgo: Big Bang vs. Entrega Continua de Incrementos

Fuente: filminas de clase (06, 08) | resumen final: páginas (16-18, 41-43)

La gestión del riesgo constituye uno de los contrastes más dramáticos de la ingeniería de software moderna:

Mitigación Tardía en Cascada (Efecto Big Bang)

Durante meses, el proyecto solo genera documentación preliminar (especificaciones, diagramas UML y planes de prueba). El riesgo tecnológico, el riesgo de usabilidad y el riesgo de integración permanecen latentes y no verificados. El momento de mayor riesgo se concentra al final del ciclo de vida, durante la fase de integración y pruebas masivas. Si un error arquitectónico o una malinterpretación funcional se detecta en esta etapa, el costo de subsanarlo se multiplica de 50 a 200 veces en comparación con una detección temprana.

Reducción Continua del Riesgo (Ciclo Ágil)

Al construir el software en rebanadas verticales que atraviesan todas las capas del sistema (interfaz, lógica y persistencia), cada iteración produce un Incremento de producto potencialmente desplegable.

La integración se realiza de forma continua (CI/CD) y las pruebas se ejecutan de manera concurrente con la codificación. El riesgo se reduce drásticamente desde las primeras semanas de trabajo gracias al feedback inmediato de usuarios reales.

Matriz Comparativa Exhaustiva: Paradigma Tradicional vs. Paradigma Ágil

Fuente: filminas de clase (01 a 06) | apunte de clase: Requerimientos Ágiles y Componentes de Proyecto | resumen final: páginas (36-41)

La siguiente matriz sistematiza de forma rigurosa los diez ejes comparativos fundamentales entre ambos paradigmas:

Dimensión de Análisis	Paradigma Tradicional (Plan-Driven)	Paradigma Ágil (Value-Driven)
Premisa Fundamental	Predictiva: El software puede planificarse completamente de antemano si se analiza con suficiente detalle.	Adaptativa y Empírica: El software es inherentemente complejo y requiere aprendizaje continuo mediante inspección y adaptación.
Manejo del Triángulo de Hierro	Alcance fijo. Tiempo y costo se estiman y varían según la dimensión del alcance.	Tiempo y costo fijos (timeboxes y equipo estable). El alcance es variable y priorizado por valor.
Ciclo de Vida Predilecto	Secuencial (Cascada, Modelo en V) o Iterativo formalizado con fases cerradas.	Iterativo-Incremental empírico en ciclos ultracortos (1 a 4 semanas).
Gestión de Requerimientos	Requerimientos exhaustivos al inicio (BRUF - <i>Big Requirements Up Front</i>). Especificación formal (ERS).	<i>Just-In-Time</i> con diferentes niveles de granularidad (Temas, Épicas, User Stories). Requerimientos emergentes.
Actitud ante el Cambio	Aversión al cambio. Se percibe como un error de planificación y se controla con burocracia (CCB, CRs).	Bienvenida al cambio. Se adopta como una ventaja competitiva y un mecanismo natural de refinamiento del producto.
Estructura de Equipos	Silos funcionales por especialidad (analistas, desarrolladores, testers) liderados por un Jefe de Proyecto.	Equipos multidisciplinarios, autónomos y autogestionados (<i>Scrum Teams</i>) sin jerarquías internas.
Relación con el Cliente	Relación contractual formal. Interacción concentrada en hitos de inicio y aceptación final.	Colaboración continua e integración directa del negocio en el equipo (rol del <i>Product Owner</i>).
Gestión de la Calidad	Fase de aseguramiento de calidad (QA/Testing) tardía y aislada, posterior a la codificación.	Calidad integrada y continua. Pruebas concurrentes, criterios de aceptación, TDD y <i>Definition of Done</i> .
Mitigación del Riesgo	Alta concentración de riesgo hacia el final del proyecto (integración masiva tardía o <i>Big Bang</i>).	Reducción temprana y sostenida del riesgo mediante incrementos funcionales desplegados en cada iteración.
Medida Principal de Éxito	Conformidad con el plan inicial (alcance, tiempo y costo aprobados en el contrato).	Software funcionando que entrega valor tangible y medible (<i>Outcome</i>) para los usuarios y el negocio.

Unidad 1: Fundamentos de la Ingeniería de Software y Gestión de Procesos

Fuente: ISW_Teorico UNIDADES 1 Y 3 (páginas 1-10) | filminas de clase (01, 06) |
apunte de clase: Componentes de Proyecto | resumen final: páginas (1-18)

Introducción a la Ingeniería del Software: Definición y Diferencia con Software

Fuente: apunte de clase: ISW_Teorico UNIDADES 1 Y 3 (página 1) | filminas de clase
(01, diapositiva 4)

PREGUNTA TÍPICA DE PARCIAL / EXAMEN FINAL

¿Cuál es la diferencia sustancial entre Software e Ingeniería de Software?

El *Software* no se reduce a mero código ejecutable; es una entidad abstracta constituida por un conjunto integrado de programas, procedimientos, documentación asociada y datos operativos necesarios para resolver un problema. La *Ingeniería de Software*, en contraste, es la disciplina de la ingeniería que aplica enfoques sistemáticos, disciplinados y cuantificables al desarrollo, operación, mantenimiento y evolución del software. Saber programar únicamente no alcanza para hacer software: se requiere comprender el dominio del negocio, negociar con los clientes y gestionar la complejidad sistémica.

Generalmente, el software es un set de programas con la documentación que los acompaña. No podemos quedarnos con la idea reduccionista de que `software = código`. Saber programar únicamente no alcanza para hacer software (hay que entender el negocio, sus requerimientos/necesidades, tratar con los clientes, establecer modelos de calidad y diseñar arquitecturas sostenibles).

Adicionalmente, al software se lo puede interpretar como **conocimiento representado en distintos niveles de abstracción**:

Requerimientos → Prototipos → Diseños → Código → Despliegue CI/CD → Ambiente de Producción

Hay un proceso de refinamiento continuo. Para construir software utilizamos el **Proceso de desarrollo de software**, el cual comprende un conjunto estructurado de etapas que van generando salidas tangibles e intangibles; esas salidas, denominadas formalmente **artefactos** en el Proceso Unificado (PUD/RUP), son software en definitiva.

El proceso de desarrollo de software clásico (como el PUD/RUP) está estructurado en flujos de trabajo (*workflows*) o disciplinas: requerimientos, análisis, diseño, implementación, prueba y despliegue. Dentro de estas etapas se ejecutan actividades concretas asignadas a roles específicos

que finalmente generan artefactos. Si únicamente consideráramos que el software es código, una inmensa masa de información crítica para la mantenibilidad y evolución del sistema quedaría descartada. Se busca definir el software con una visión integradora y englobadora.

Existen 3 tipos básicos de software en el ecosistema informático:

1. **System Software (Software de Sistema):** Software de base encargado de controlar, coordinar y gestionar los recursos del hardware de una computadora, proporcionando además la plataforma operativa e interfaz para que interactúen los usuarios y otras aplicaciones (ej. sistemas operativos, controladores de dispositivos).
2. **Utilitarios:** Subconjunto de software que brinda herramientas operativas adicionales para optimizar, diagnosticar, mantener o proteger el funcionamiento del sistema operativo y los componentes físicos (ej. antivirus, compresores de datos, gestores de disco).
3. **Software de Aplicación:** Programas diseñados para permitir a los usuarios finales realizar tareas específicas, productivas o comerciales. Se enfoca directamente en satisfacer las necesidades funcionales del negocio y de los usuarios (ej. sistemas de facturación, navegadores web, suites ofimáticas).

Naturaleza del Software vs. Manufactura Tradicional

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 1-2) | filminas de clase (01, diapositivas 5-7)

PRINCIPIO FUNDAMENTAL: EL SOFTWARE NO ES MANUFACTURA

No hay manera rigurosa de comparar el desarrollo de software con una línea de producción industrial o de manufactura física. Tratar al software como una cadena de montaje fabril es el origen de los mayores fracasos en la gestión de proyectos informáticos.

Las razones teóricas y empíricas por las cuales el software difiere radicalmente de la manufactura tradicional son las siguientes:

- **El software es menos predecible:** A diferencia de los productos tangibles en una línea de montaje (donde las tolerancias mecánicas y las leyes de la física establecen comportamientos predeterminados), el software se desenvuelve en entornos abstractos y variables.
- **No hay producción en masa:** Casi ningún producto de software es idéntico a otro. Los requerimientos, regulaciones, integraciones y expectativas de los usuarios son siempre particulares para cada organización y contexto. En software, el costo principal radica en la ingeniería, el diseño y la construcción inicial; el costo de "fabricación" o duplicación de copias físicas/digitales es prácticamente nulo.
- **No todas las fallas son errores de diseño:** En la manufactura, una falla suele originarse en el desgaste de un material o un defecto en la línea de montaje. En el software, muchas fallas

emergen de interacciones imprevistas entre componentes válidos, cambios en el entorno operativo de ejecución o desajustes respecto a las expectativas cambiantes de los usuarios.

- **El software no se desgasta por uso físico:** El hardware experimenta fricción, calor y fatiga material, describiendo la clásica "Curva de la Bañera" (*Bathtub Curve*), donde la tasa de fallas es alta al inicio (mortalidad infantil), se estabiliza durante la vida útil y se dispara al final por envejecimiento físico. El software, en cambio, no sufre desgaste mecánico. Sin embargo, su tasa de fallas real suele incrementarse a lo largo del tiempo debido a la acumulación de cambios, parches de mantenimiento no estructurados y evolución del entorno operativo, un fenómeno conocido como **degradación o entropía del software**.

La Crisis del Software y sus Causas Fundamentales

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 2-3) | filminas de clase (01, diapositivas 8-11)

El término **Crisis del Software** fue formalmente acuñado en la conferencia fundacional de la OTAN celebrada en **Garmisch (Alemania) en 1968**. Dicha conferencia congregó a los mayores expertos mundiales para debatir un problema alarmante: la incapacidad sistemática de la industria informática para construir sistemas de software confiables dentro de los plazos y presupuestos previstos.

Con el avance vertiginoso del hardware en la década de 1960 (computadoras más potentes, memorias más amplias y procesadores más rápidos), creció de forma exponencial la demanda de aplicaciones más complejas. La capacidad de construir hardware superó por amplio margen la capacidad teórica y metodológica de diseñar software confiable.

Las causas fundamentales que originaron y sostienen la crisis del software son:

- **Crecimiento desmedido de la complejidad:** Los sistemas demandados dejaron de ser programas de cálculo aislados para convertirse en soluciones corporativas distribuidas y concurrentes.
- **Costos desfasados de mantenimiento:** La industria descubrió tardíamente que la inmensa mayoría de los costos de un software no se consumen durante el desarrollo inicial, sino en su operación y modificación a largo plazo. Un software mal diseñado desde el inicio genera costos de mantenimiento insostenibles.
- **Fallas de comunicación interdisciplinaria:** Dificultad intrínseca para traducir las necesidades de negocio de los clientes en especificaciones técnicas inequívocas para los programadores.
- **Falta de rigor metodológico:** Ausencia histórica de estándares, métricas de calidad y procesos formales de ingeniería, considerándose el desarrollo como una labor artesanal dependiente exclusivamente del talento de programadores individuales.
- **Baja confiabilidad y fallas operativas:** Entrega de productos con altos índices de defectos ocultos que provocaban pérdidas económicas masivas e insatisfacción crónica en los usuarios

finales.

El Paper "No Silver Bullet": Dificultades Esenciales vs. Accidentales

Fuente: apunte de clase: ISW_Teorico UNIDADES 1 Y 3 (páginas 3-6) | filminas de clase (01, diapositivas 12-16)

En 1986, **Frederick P. Brooks Jr.** publicó su célebre ensayo "*No Silver Bullet — Essence and Accident in Software Engineering*". La premisa de Brooks se inspira en el folklore popular: así como no existe una "bala de plata" mágica capaz de exterminar de un solo disparo al hombre lobo, **no existe ningún avance tecnológico, metodológico o de gestión individual que por sí solo prometa un incremento de un orden de magnitud (10x) en la productividad, confiabilidad o simplicidad del software en una década.**

Brooks dividió las dificultades inherentes al desarrollo de software en dos dimensiones conceptuales fundamentales:

1. Dificultades Accidentales

Son aquellos obstáculos derivados de la imperfección de las herramientas técnicas utilizadas para plasmar el software en una época determinada. Pertenecen al dominio de la implementación tecnológica (el "cómo" materializarlo).

Superación Histórica: La ingeniería resolvió progresivamente las dificultades accidentales mediante:

- **Lenguajes de alto nivel (Fortran, C, Java, Python):** Liberaron al programador del pensamiento a nivel de registros de procesador y código máquina binario.
- **Sistemas de tiempo compartido:** Eliminaron las esperas de horas o días del procesamiento por lotes (*batch*), permitiendo una interacción directa e inmediata con el código.
- **Entornos de Desarrollo Integrados (IDEs):** Centralizaron edición, compilación, depuración visual y navegación de archivos.

2. Dificultades Esenciales

Constituyen la naturaleza intrínseca, irreductible y conceptual del software: la formulación abstracta del entramado lógico de conceptos interconectados. Brooks determinó que estas dificultades no pueden eliminarse porque son la esencia misma de lo que significa crear software.

Las Cuatro Características de la Esencia del Software

Fuente: apunte de clase: ISW_Teorico UNIDADES 1 Y 3 (páginas 4-5) | filminas de clase (01, diapositivas 12-14) Brooks identificó cuatro propiedades intrínsecas que componen la dificultad esencial:

1. **Complejidad:** Los sistemas de software son más complejos para su tamaño que cualquier otra construcción humana, porque no existen dos piezas idénticas a nivel macro. Si dos módulos son exactamente iguales, se consolidan en una única función o librería. A medida que un sistema escala, sus elementos interactúan entre sí de manera exponencial y no lineal, dificultando el control cognitivo del desarrollador.
2. **Conformidad:** El software no está gobernado por leyes físicas naturales unificadas (como la gravedad en ingeniería civil). Debe conformarse y adaptarse forzosamente a interfaces humanas arbitrarias, regulaciones legales cambiantes, instituciones burocráticas y sistemas heredados (*legacy*) preexistentes.
3. **Cambiabilidad (Mutabilidad):** Todo software exitoso está sometido a una presión continua de modificación. Existen dos razones inevitables: a medida que el software demuestra su utilidad, los usuarios descubren nuevos usos y solicitan más capacidades; y en paralelo, el software debe sobrevivir a la evolución constante de su hardware y entorno operativo de base.
4. **Invisibilidad:** El software es inherentemente geométrico e invisible en el espacio. No puede representarse adecuadamente mediante diagramas bidimensionales o tridimensionales (como los planos de un arquitecto). Cuando se intenta diagramar el flujo de control, se pierde la estructura de datos; cuando se visualiza la herencia, se ocultan las interacciones dinámicas en tiempo de ejecución.

Ataques a la Esencia Propuestos por Brooks

Fuente: apunte de clase: ISW_Teorico UNIDADES 1 Y 3 (páginas 5-6) | filminas de clase (01, diapositiva 15) Dado que las dificultades accidentales han sido ampliamente resueltas, Brooks propuso cuatro estrategias para mitigar el núcleo de las dificultades esenciales:

- **Comprar vs. Construir (Buy vs. Build):** La forma más barata y rápida de construir software es no construirlo. Reutilizar paquetes comerciales o librerías existentes elimina por completo la necesidad de diseñar la conceptualización interna.
- **Refinamiento de Requerimientos y Prototipado Rápido:** Dado que el cliente no sabe con precisión qué necesita hasta que ve una versión funcional, el prototipado iterativo ayuda a estabilizar la estructura conceptual esencial antes de la construcción a gran escala.
- **Cultivar en lugar de Construir (Grow, not Build):** Adoptar un enfoque de desarrollo orgánico e incremental. Diseñar un sistema esqueleto mínimo que compile y ejecute, e ir agregándole funcionalidad funcional paso a paso.
- **Grandes Diseñadores:** Las metodologías y herramientas disciplinan y potencian al profesional promedio, pero los saltos cualitativos en la arquitectura de software provienen del talento, intuición y diseño de profesionales excepcionales. Las organizaciones deben cultivar y retener a estos talentos como su activo más crítico.

Disciplinas de la Ingeniería de Software y Evolución del Enfoque Sistémico

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 6-7) | filminas de clase (01, diapositivas 14-16)

Para contrarrestar la crisis del software y gestionar las dificultades esenciales, la ingeniería de software moderna se organiza en un cuerpo sistemático de disciplinas técnicas y de gestión:



Procesos de Desarrollo de Software: Definidos vs. Empíricos

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 7-9) | filminas de clase (01, diapositivas 17-22)

Un **Proceso de Software** se define formalmente como un conjunto estructurado de actividades, acciones, tareas, roles y artefactos requeridos para construir, desplegar y mantener un producto de software. Existen dos escuelas fundamentales en la teoría de procesos:

1. Procesos Definidos (Industriales / Predictivos):

- Proviene del control estadístico de procesos de la ingeniería química y manufacturera clásica.
- **Premisa Teórica:** Asumen que cada problema de desarrollo puede descomponerse en una secuencia perfectamente comprendida y predecible de pasos estandarizados. Si las entradas se especifican correctamente y se aplican las mismas actividades y procedimientos, los resultados serán uniformes y predecibles, independientemente de quién ejecute el trabajo.

- **Mecanismo de Control:** Se basan en el control preventivo y la planificación anticipada. Los cambios representan anomalías que deben regularse estrictamente.
- **Ejemplos Canónicos:** Proceso Unificado de Desarrollo (PUD), Rational Unified Process (RUP) en su formulación prescriptiva tradicional.

2. Procesos Empíricos (Adaptativos / Ágiles):

- Reconocen que el desarrollo de software es una **actividad de conocimiento no lineal y creativa** inmersa en una constante incertidumbre.
- **Premisa Teórica:** No es posible extrapolar linealmente la experiencia de un proyecto a otro ni planificar todo de antemano. Los procesos empíricos operan mediante la experimentación activa, el ensayo y error controlado, y el aprendizaje acumulado por el propio equipo en cada ciclo de trabajo.
- **Mecanismo de Control:** La administración se efectúa mediante inspecciones frecuentes y adaptaciones continuas. No se busca eliminar la incertidumbre antes de comenzar, sino descubrirla e interactuar con ella mediante software funcional entregado en plazos mínimos.
- **Ejemplos Canónicos:** Scrum, Lean Software Development, Kanban, Extreme Programming (XP).

Cuadro Comparativo Exhaustivo: Procesos Definidos vs. Procesos Empíricos

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 8-9) | filminas de clase (01, diapositiva 21) | resumen final: páginas (14-16)

La siguiente tabla comparativa sintetiza las dimensiones estructurales que diferencian a los Procesos Definidos de los Procesos Empíricos:

Dimensión de Análisis	Procesos Definidos (PUD / RUP Tradicional)	Procesos Empíricos (Agile / Lean / Scrum)
Base Teórica y Origen	Ingeniería industrial, manufactura y control estadístico de procesos repetibles.	Teoría de la complejidad, empirismo científico y aprendizaje organizacional interactivo.
Premisa sobre el Trabajo	Todo el sistema puede especificarse, comprenderse y planificarse antes de la construcción.	El software es complejo e impredecible; los requerimientos se descubren mediante el uso real.
Predictibilidad y Repetibilidad	Alta predictibilidad teórica. Busca que el proceso sea repetible con independencia de los ejecutores.	Predictibilidad empírica basada en velocidad histórica demostrada por el equipo en acción.
Extrapolación de Experiencia	Asume que las métricas y estimaciones de proyectos previos son directamente extrapolables.	Las experiencias no son linealmente extrapolables; cada ecosistema y producto es único.
Actitud ante el Cambio	El cambio se percibe como una desviación nociva y se gestiona mediante control burocrático (CCB).	El cambio se abraza como un factor competitivo natural; los procesos facilitan la adaptación rápida.
Mecanismo Central de Control	Control por seguimiento de planes iniciales, cronogramas y fases formalmente concluidas.	Control por inspección frecuente y adaptación continua en ciclos cortos de retroalimentación.
Ciclos de Vida Aplicables	Secuenciales (Cascada, V) o Iterativos-Incrementales formales con hitos cerrados.	Exclusivamente ciclos iterativos ultracortos. Se prohíbe el uso de ciclos secuenciales puros.
Rol de la Documentación	Documentación exhaustiva previa a la codificación (contratos, ERS, diseños detallados).	Documentación viva, concisa y suficiente (<i>Just Enough</i>). El software funcional es la evidencia primaria.
Liderazgo y Equipos	Silos jerárquicos comandados por un Jefe de Proyecto que asigna y controla tareas individuales.	Equipos multidisciplinarios, empoderados, autónomos y autogestionados sin jefaturas directas.
Manejo del Conocimiento	Basado en conocimiento explícito codificado en artefactos formales transferidos entre etapas.	Transforma el conocimiento tácito en explícito mediante transparencia colectiva y diálogo cara a cara.

Los Tres Pilares del Empirismo en el Software

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 9-10) | filminas de clase (01, 08)

Todo proceso empírico de desarrollo de software se sustenta rigurosamente sobre tres pilares interdependientes:

1. Transparencia

Todos los aspectos significativos del proceso deben ser visibles y comprensibles tanto para quienes realizan el trabajo como para quienes lo reciben.

Permite transformar el conocimiento implícito de cada individuo en conocimiento explícito del equipo. Los artefactos que carecen de transparencia conducen a decisiones defectuosas, aumentan el riesgo y generan desperdicio. **La transparencia es el prerequisite ineludible de la inspección: la inspección sin transparencia genera engaño.**

2. Inspección

El equipo examina con frecuencia los artefactos generados y el progreso del trabajo hacia los objetivos acordados para detectar desviaciones o problemas indeseables.

La inspección no debe ser una auditoría burocrática paralizante, sino un examen técnico ágil realizado por los mismos creadores del producto. **La inspección habilita la adaptación: inspeccionar sin estar dispuestos a cambiar no tiene sentido operativo.**

3. Adaptación

Si la inspección determina que uno o más aspectos del proceso se desvían de los límites aceptables o que el producto resultante no satisface el valor esperado, el proceso o el material se ajustan de inmediato.

La adaptación materializa la mejora continua. Se requiere que los integrantes del equipo estén facultados para tomar decisiones y ajustar su dinámica de trabajo sin requerir autorizaciones burocráticas lentas.

Modelos de Proceso y Ciclos de Vida del Software

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 10) | filminas de clase (01, diapositivas 23-30)

DEFINICIÓN SAGRADA DE CÁTEDRA

Se denomina **Ciclo de Vida** a una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Un ciclo de vida de un proyecto de software es una **representación simplificada de un proceso**. Grafica una descripción del proceso desde una perspectiva particular y **define cómo usar el proceso en el contexto del proyecto**.

Mientras el *proceso* define genéricamente qué actividades hay que ejecutar, qué roles intervienen y qué artefactos se generan, el *ciclo de vida del proyecto* establece el orden temporal, la frecuencia

con la que se ejecutan dichas actividades y la ubicación estratégica de los puntos de control y revisión.

Existen tres arquetipos esenciales de ciclos de vida:

1. **Secuencial:** Las actividades se ejecutan en estricto orden cronológico lineal. Cada fase debe estar 100% completada y aprobada antes de iniciar la subsiguiente.
2. **Iterativo:** El proyecto se divide en ciclos repetitivos donde se ejecutan parcialmente todas las disciplinas de desarrollo, refinando y ampliando el producto en cada ciclo.
3. **Rekursivo / Evolutivo:** Las estructuras de proceso se aplican de forma autorreferencial sobre incrementos sucesivos, permitiendo que la arquitectura y los requerimientos emerjan de la retroalimentación continua.

Análisis Comparativo de Modelos de Ciclos de Vida

Fuente: filmas de clase (01, diapositivas 24-30) | resumen final: páginas (1-14)

1. Modelo en Cascada Clásico (Waterfall)

Fuente: filmas de clase (01, diapositivas 24-25) | resumen final: páginas (2-4)
Propuesto formalmente por Winston Royce en 1970 como base analítica. Su estructura es estrictamente lineal y descendente: Requerimientos → Análisis → Diseño → Codificación → Pruebas → Operación.

- **Ventajas:** Estructura altamente intuitiva y disciplinada. Facilita la asignación contractual de responsabilidades y la medición de avances basada en documentos terminados en cada hito.
- **Desventajas:** Inflexibilidad absoluta ante cambios. La integración tardía genera el temido "efecto Big Bang", donde los defectos mayores se descubren cuando el costo de corrección es máximo. El cliente solo ve software operativo al final del calendario del proyecto.
- **Aplicabilidad Recomendada:** Exclusivamente en proyectos con requerimientos 100% conocidos, estables y congelados, con tecnología probada y bajo nivel de incertidumbre.

2. Modelo en V (Verificación y Validación)

Fuente: filmas de clase (01, diapositiva 26) | resumen final: páginas (4-6) Variante de ingeniería del modelo en cascada que asocia explícitamente a cada fase descendente de desarrollo su correspondiente fase ascendente de prueba:

- Requerimientos de Usuario ↔ Pruebas de Aceptación.
- Requerimientos de Sistema ↔ Pruebas de Sistema.
- Diseño de Arquitectura ↔ Pruebas de Integración.
- Diseño Detallado / Módulos ↔ Pruebas Unitarias.

- **Ventajas:** Eleva la visibilidad del testing desde las primeras etapas y formaliza la trazabilidad de verificación.
- **Desventajas:** Mantiene la misma rigidez secuencial que la cascada frente a cambios en los requerimientos.

3. Modelo Prototipado (Descartable vs. Evolutivo)

Fuente: filminas de clase (01, diapositiva 27) | resumen final: páginas (6-8) Se fundamenta en la construcción de representaciones operativas preliminares del software para resolver la ambigüedad en los requerimientos del cliente:

- **Prototipo Rápido / Descartable (Throwaway):** Su único fin es explorar y validar requerimientos o interfaces de usuario con el cliente. Una vez aclarado el objetivo, el prototipo se desecha y el sistema definitivo se construye desde cero bajo estrictos estándares.
- **Prototipo Evolutivo:** Comienza con un subconjunto inicial de requerimientos comprendidos y, a través de iteraciones continuas y refactorizaciones sucesivas, el prototipo madura gradualmente hasta convertirse en el sistema productivo final.

4. Modelo Iterativo-Incremental (PUD / RUP)

Fuente: filminas de clase (01, diapositivas 28-29) | resumen final: páginas (8-10) El sistema se desarrolla mediante una sucesión planificada de mini-proyectos denominados *iteraciones*. Cada iteración atraviesa las disciplinas esenciales de ingeniería (análisis, diseño, código y pruebas) y entrega un **Incremento**: un subconjunto funcional y ejecutable del software.

- **Fases del Proceso Unificado:** Incepción (definir visión y alcance), Elaboración (mitigar riesgos principales y estabilizar la arquitectura), Construcción (desarrollar funcionalidad masiva) y Transición (despliegue en producción y validación de usuarios).
- **Ventajas:** Permite incorporar requerimientos emergentes, estabiliza tempranamente la arquitectura y ofrece visibilidad constante del avance real mediante versiones operativas.

5. Modelo en Espiral de Barry Boehm

Fuente: filminas de clase (01, diapositiva 30) | resumen final: páginas (10-12) Diseñado para proyectos a gran escala de alta criticidad. Cada revolución de la espiral recorre cuatro cuadrantes de forma sistemática:

1. **Determinar Objetivos:** Identificar metas, alternativas y restricciones.
2. **Evaluar Alternativas y Gestionar Riesgos:** Identificar y mitigar proactivamente los riesgos técnicos y operativos (análisis de riesgos mediante simulación, prototipado o pruebas de concepto).
3. **Desarrollar y Verificar:** Construir el producto o incremento correspondiente al nivel de madurez de la fase.

4. **Planificar la Siguiente Fase:** Revisar los resultados y definir los objetivos de la siguiente espiral con los involucrados.

- **Ventajas:** Es el modelo con mayor foco en la gestión integral de riesgos de toda la ingeniería de software.
- **Desventajas:** Elevado costo de gestión, requiere una alta pericia técnica en evaluación de riesgos por parte del equipo directivo.

Criterios de Selección del Modelo de Ciclo de Vida

Fuente: filminas de clase (01, diapositivas 31-35) | resumen final: páginas (12-14)

La selección del modelo de ciclo de vida más adecuado para un proyecto debe fundamentarse en una evaluación rigurosa de las siguientes variables de contexto:

Criterio de Decisión	Modelos Secuenciales (Cascada / V)	Modelos Iterativos / Ágiles (Scrum / PUD)
Claridad de Requerimientos	Requerimientos estables, completamente claros y definidos desde el inicio.	Requerimientos emergentes, ambiguos o sujetos a evolución constante.
Estabilidad Tecnológica	Tecnologías conocidas, dominadas por el equipo y ampliamente maduras.	Tecnologías novedosas, en evolución o con necesidad de validación experimental.
Involucramiento del Cliente	El cliente interviene únicamente al inicio (especificación) y al final (aceptación).	El cliente o sus representantes colaboran de manera diaria o continua en el proyecto.
Tolerancia al Riesgo	Baja tolerancia a cambios imprevistos; entorno contractualmente rígido.	Alta necesidad de mitigación temprana de riesgos mediante entregas funcionales.
Tamaño y Distribución	Proyectos con especificaciones legales fijas y grandes equipos subcontratados.	Equipos autónomos de hasta 10 personas enfocados en entrega rápida de valor.

Componentes de un Proyecto de Desarrollo de Software y las 4 P's

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositivas 2-15) | resumen final: páginas (14-18)

Un **Proyecto de Software** es un esfuerzo temporal y organizado destinado a crear un producto, servicio o resultado de software único. Posee cuatro características formales determinantes:

1. **Es Único:** El resultado final no puede producirse masivamente por medio de una línea de producción seriada. Cada desarrollo enfrenta particularidades técnicas, funcionales y de negocio irrepetibles.
2. **Duración Limitada:** Tiene una fecha de inicio y una fecha de finalización estrictamente definidas en su concepción.
3. **Objetivo Claro y Alcanzable:** Debe tener definida una meta precisa que sea comprensible para todo el equipo y viable de alcanzar con los recursos asignados. No se admiten objetivos ambiguos.
4. **Tareas Interrelacionadas Basadas en Recursos y Esfuerzo:** La complejidad se gestiona descomponiendo el trabajo en actividades más pequeñas y manejables, cuyos esfuerzos combinados conducen al resultado final.

Las 4 P's del Software

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositiva 4) Toda gestión eficaz de proyectos de software se articula en torno a cuatro dimensiones interdependientes:

- **Personas (People):** Es el componente más crítico e influyente. El éxito de un proyecto depende de las habilidades técnicas, la motivación, la comunicación y la dinámica colaborativa de los profesionales involucrados.
- **Producto (Product):** La solución de software a desarrollar. Incluye el alcance del producto, los requerimientos funcionales y no funcionales, y los objetivos de negocio a satisfacer.
- **Proceso (Process):** El marco metodológico y conjunto estructurado de disciplinas, actividades, roles y tareas que guían el ciclo de trabajo de ingeniería.
- **Proyecto (Project):** La planificación, seguimiento, monitoreo y asignación de recursos necesarios para gobernar la ejecución del proceso y entregar el producto conforme a las restricciones acordadas.

Leyes Fundamentales de Fred Brooks y Dinámica de Proyectos

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositivas 6-10)

La Ley de Brooks

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositivas 6-8) En su obra clásica "*The Mythical Man-Month*", Fred Brooks formuló una de las máximas más célebres de la disciplina:

LEY DE BROOKS

"Añadir personal a un proyecto de software con retraso solo conseguirá retrasarlo más."

Este fenómeno ocurre por dos razones fundamentales:

1. **Rampa de Aprendizaje y Transferencia:** Los ingenieros recién incorporados no son productivos de inmediato. Para aprender la arquitectura y el código, deben ser capacitados por los desarrolladores más experimentados del equipo, deteniendo o ralentizando la producción de quienes ya estaban avanzando.
2. **Explosión Combinatoria de Canales de Comunicación:** En una tarea indivisible o de conocimiento complejo, cada nueva persona añadida incrementa de forma no lineal los canales de comunicación interpersonal requeridos para coordinar el trabajo:

$$\text{Canales de Comunicación} = n \times (n - 1) / 2$$

Donde n representa el número total de personas involucradas. Pasar de 5 a 10 ingenieros no duplica las interacciones: incrementa los canales de 10 a 45, disipando el esfuerzo en reuniones de coordinación, sincronización y alineación.

El Efecto del Segundo Sistema (Second-System Effect)

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositiva 9) Brooks identificó un patrón de comportamiento recurrente en los arquitectos de software: el sistema más peligroso de diseñar es siempre el segundo. Al diseñar el primer producto, el ingeniero es disciplinado y cauteloso debido a su inexperiencia. Tras el éxito de ese primer sistema, el arquitecto acomete el segundo proyecto lleno de confianza y tiende a sobrecargarlo incorporando todas las ideas, funciones accesorias y complejidades que no pudo incluir en el primero, desembocando en un sistema sobrecargado, ineficiente y propenso al colapso.

Vínculo Sistémico: Proceso - Proyecto - Producto

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filminas de clase (06, diapositivas 11-14)

La relación entre estas tres entidades constituye la base de la administración en ingeniería de software:

- **El Proceso** define la teoría abstracta de cómo se construye el software: establece las mejores prácticas, las disciplinas de ingeniería, los roles genéricos y los tipos de artefactos que deben

producirse. Responde a la pregunta: *¿Qué actividades y técnicas son necesarias para hacer software de calidad?*

- **El Proyecto** es el vehículo temporal de gestión que aplica el proceso en un contexto corporativo particular. Fija el cronograma, asigna los recursos humanos reales, gestiona los costos y selecciona el ciclo de vida apropiado. Responde a la pregunta: *¿Quién, cuándo, en qué orden y con qué presupuesto se ejecutará el trabajo?*
- **El Producto** es el resultado técnico y tangible que emerge como salida de la ejecución del proyecto. A diferencia del proyecto (que tiene un fin temporal estricto), **el producto sobrevive al proyecto**: entra en fase operativa, genera valor en el negocio, requiere mantenimiento y evoluciona a lo largo de los años.

DISTINCIÓN CRUCIAL PARA EXAMEN

Alcance del Producto vs. Alcance del Proyecto:

* El **Alcance del Producto** se especifica en la **Especificación de Requerimientos de Software (ERS)** o en el **Product Backlog**. Define las características, funcionalidades y requerimientos no funcionales que debe contener la solución de software.

* El **Alcance del Proyecto** se documenta en el **Plan de Proyecto**. Define todo el trabajo que el equipo debe realizar para entregar el producto terminado, incluyendo actividades de gestión, creación del plan de SCM, configuración de entornos de prueba, capacitaciones a usuarios y despliegue.

Proyectos Exitosos vs. Proyectos Fallidos: Métricas y Lecciones

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filmas de clase (06, diapositivas 15-17) | CHAOS Report de Standish Group

El análisis sistemático del desempeño en la industria informática internacional, popularizado por los estudios históricos del **CHAOS Report del Standish Group**, clasifica los proyectos de desarrollo en tres categorías:

1. **Exitosos (Successful):** Entregados a tiempo, dentro del presupuesto asignado y con todas las funcionalidades comprometidas funcionando con la calidad requerida.
2. **Desafiados / Con Desvíos (Challenged):** Completados y puestos en operación, pero sobrepasando significativamente los plazos temporales, con sobrecostos presupuestarios o con recortes severos en el alcance pactado.
3. **Fallidos / Cancelados (Failed):** Proyectos cancelados antes de su finalización o que, al entregarse, resultaron completamente inutilizables, representando una pérdida total de la inversión.

Causas Primarias de Fracaso en Proyectos de Software

Fuente: apunte de clase: Componentes de Proyecto de Desarrollo de SW | filmas de clase (06, diapositivas 16-17) | CHAOS Report Las investigaciones empíricas demuestran de manera consistente que los fracasos en el software rara vez se originan en limitaciones puramente técnicas de hardware o algoritmos. Las causas raíces se concentran en factores humanos y de gestión:

- Requerimientos incompletos o mal comprendidos al inicio del ciclo de vida.
 - Falta de involucramiento continuo y activo por parte de los usuarios finales y responsables del negocio.
 - Expectativas desalineadas e irreales impuestas por la dirección ejecutiva.
 - Cambios descontrolados de alcance sin ajuste proporcional de recursos o tiempos (*scope creep*).
 - Planificación y estimación determinista fundamentada en cronogramas ficticios sin base empírica.
-

Unidad 2: Gestión de Producto, Requerimientos Ágiles, Estimaciones y Scrum

Fuente: filmas de clase (03, 04, 05, 07, 08) | apuntes de clase: Requerimientos Ágiles, Estimaciones y Scrum | resumen final: páginas (36-70, 81-90)

Módulo 2.1: Gestión Estratégica de Producto de Software

Fuente: filmas de clase (05, diapositivas 1-32) | resumen final: páginas (81-90)

Filosofía de Producto vs. Proyecto: Del Enfoque Temporal a la Solución Viva

Fuente: filmas de clase (05, diapositivas 2-6) | resumen final: páginas (81-83)

En la ingeniería moderna, la transición desde la "Gestión de Proyectos" hacia la "Gestión de Productos" representa un salto cualitativo estratégico:

- **El Proyecto de Software** es un vehículo temporal con un inicio y fin fijados de antemano. Su objetivo fundamental es la ejecución disciplinada del plan para producir una entrega comprometida dentro de las restricciones de tiempo y presupuesto. Una vez entregado el sistema, el proyecto finaliza y el equipo se disuelve.
- **El Producto de Software** es la solución viva, continua y evolutiva que emerge de los proyectos y persiste en el mercado. Un producto no es un mero conjunto estático de código;

es un activo estratégico que resuelve problemas de usuarios, genera ingresos y evoluciona a lo largo de un ciclo de vida propio que incluye concepción, crecimiento, maduración y eventualmente obsolescencia o retiro.

DIFERENCIA FUNDAMENTAL: OUTPUT VS. OUTCOME

* **Output (Salida):** Es el artefacto físico o digital producido por el equipo de desarrollo (líneas de código compiladas, funcionalidades construidas, pantallas diseñadas, casos de prueba ejecutados). Medir el éxito solo por output conduce a la "trampa de construir más para entregar menos valor".

* **Outcome (Resultado / Beneficio):** Es el impacto real y medible que genera el software en el comportamiento de los usuarios o en las métricas del negocio (ej. reducir en 30% el tiempo de atención al cliente, incrementar la tasa de retención o disminuir errores transaccionales). La gestión de productos ágil prioriza maximizar el *Outcome* minimizando el *Output* necesario para alcanzarlo.

El Enfoque Lean Startup: Ciclo Crear - Medir - Aprender y Validación de Hipótesis

Fuente: filminas de clase (05, diapositivas 7-10) | resumen final: páginas (83-84)

Inspirado por Eric Ries, el desarrollo ágil de productos adopta el principio de que todo nuevo desarrollo se fundamenta inicialmente en **hipótesis de negocio no comprobadas** (supuestos sobre qué necesita el cliente y si estará dispuesto a utilizarlo o pagarlo).

El núcleo metodológico es el circuito de retroalimentación **Crear - Medir - Aprender (Build - Measure - Learn)**:

1. **Ideas e Hipótesis:** Formulación de supuestos comprobables sobre las necesidades del mercado.
2. **Crear:** Construcción de la versión más pequeña y funcional del producto capaz de poner a prueba la hipótesis con el mínimo esfuerzo posible (el MVP).
3. **Medir:** Recolección de datos empíricos cuantitativos y cualitativos sobre cómo interactúan los usuarios reales con dicha solución.
4. **Aprender:** Análisis riguroso de las métricas obtenidas para validar o refutar la hipótesis original, tomando una decisión estratégica crítica:
 - **Perseverar:** La hipótesis se valida; se profundiza la inversión y el desarrollo en esa dirección funcional.
 - **Pivote:** La hipótesis resulta falsa; se modifica de forma sustancial la estrategia, el modelo de valor o el segmento de usuarios sin cambiar la visión global del producto.

El Espectro de los Productos Mínimos: MVP vs. MMP vs. MLP

Fuente: filminas de clase (05, diapositivas 11, 21-25) | apunte de clase: Requerimientos Ágiles | resumen final: páginas (88-90)

La cátedra establece una clara jerarquía entre las diferentes formulaciones de productos mínimos:

1. MVP (Minimum Viable Product)

Objetivo Central: Aprendizaje Validado.

Es la versión más básica de una solución que permite al equipo recopilar la máxima cantidad de aprendizaje validado sobre los clientes con el menor esfuerzo de desarrollo.

Anatomía de Cátedra: El MVP debe ser una **rebanada vertical funcional** que atravesara todas las capas (interfaz de usuario mínima, lógica operativa esencial y persistencia básica). No es aceptable construir solo una capa horizontal desconectada (ej. solo el diseño de la base de datos o maquetas gráficas sin interacción real).

2. MMP (Minimum Marketable Product) / MMR

Objetivo Central: Retorno Comercial y Adopción.

Conocido también como *Minimum Marketable Release (MMR)*. Es la versión más pequeña de un producto que satisface un conjunto suficiente de necesidades de los usuarios como para ser comercializada, vendida y adoptada de forma viable en un mercado real, generando valor económico inmediato.

3. MLP (Minimum Lovable Product)

Objetivo Central: Deleite y Fidelización Emocional.

Evolución moderna del MVP que no solo busca la viabilidad técnica y funcional básica, sino que incorpora una experiencia de usuario (UX) pulida, diseño empático y deleite emocional suficiente para enamorar a los primeros adoptantes (*early adopters*) y diferenciarse en mercados competitivos.

Dual-Track Agile: Product Discovery vs. Product Delivery

Fuente: filmas de clase (05, diapositivas 12-14) | resumen final: páginas (85-87)

El modelo de doble vía (*Dual-Track Agile*) concibe el trabajo ágil como dos corrientes continuas y paralelas que se alimentan mutuamente:

Vía 1: Product Discovery (Descubrimiento)

Propósito: Construir el Producto Correcto.

Enfocado en explorar problemas de usuarios, formular hipótesis, prototipar rápidamente y mitigar los riesgos de valor (¿lo querrán?), usabilidad (¿sabrán usarlo?), factibilidad técnica (¿podemos construirlo?) y viabilidad comercial (¿funciona para nuestro negocio?). Su salida son Historias de Usuario validadas que cumplen con la Definición de Listo (DoR).

Vía 2: Product Delivery (Entrega)

Propósito: Construir el Producto Correctamente.

Enfocado en la ingeniería de software rigurosa, codificación, aseguramiento de la calidad, testing automatizado y despliegue continuo en producción. Toma los requerimientos validados en la vía de Discovery y los transforma en incrementos de software robustos y potencialmente desplegables que cumplen la Definición de Hecho (DoD).

Design Thinking y el Esquema del Doble Diamante

Fuente: filmas de clase (05, diapositivas 26-27) | resumen final: páginas (84-85)

El diseño centrado en el usuario se formaliza mediante el esquema del **Doble Diamante**, que alterna entre pensamiento divergente (explorar ampliamente) y convergente (tomar decisiones enfocadas):

- **Diamante 1: Definir el Problema Correcto**

- *Descubrir (Divergencia)*: Empatizar con los usuarios reales, realizar entrevistas de campo y observar problemas sin juzgar de antemano la solución.
- *Definir (Convergencia)*: Sintetizar la investigación empírica para formular el problema central y las necesidades insatisfechas a resolver.

- **Diamante 2: Construir la Solución Correcta**

- *Desarrollar (Divergencia)*: Idear múltiples alternativas de solución, diseñar prototipos preliminares y experimentar con escenarios de interacción.
- *Entregar (Convergencia)*: Seleccionar la alternativa de mayor viabilidad y valor, pulir los detalles funcionales y preparar los requerimientos para el desarrollo formal.

Visión del Producto y Plantilla de Elevator Pitch

Fuente: filminas de clase (05, diapositiva 28) | resumen final: páginas (86-87)

La **Visión del Producto** es el faro estratégico que guía a todos los integrantes del equipo hacia un propósito común a largo plazo. Define qué es el producto, para quién se construye y por qué es superior a las alternativas del mercado.

Para estructurarla de forma rigurosa, la cátedra adopta la célebre plantilla de **Geoffrey Moore**:

PLANTILLA CANÓNICA DE ELEVATOR PITCH (GEOFFREY MOORE)

Para [cliente o segmento objetivo]

Que [tiene una necesidad insatisfecha o problema crítico a resolver],

El producto [nombre del producto de software]

Es una categoría de [categoría o tipo de solución de software]

Que [beneficio clave principal o propuesta de valor diferenciadora].

A diferencia de [competidor principal o alternativa analógica actual],

Nuestro producto [diferenciador competitivo o ventaja tecnológica sustancial].

Product Roadmaps: De Roadmaps de Características a Roadmaps Basados en Resultados

Fuente: filminas de clase (05, diapositivas 29-32) | resumen final: páginas (87-88)

En la gestión tradicional, el roadmap era un cronograma rígido tipo diagrama de Gantt repleto de funcionalidades comprometidas con fechas fijas de entrega a meses o años vista. En entornos ágiles, esta práctica es considerada un generador masivo de desperdicio y falsas expectativas.

El **Roadmap Ágil Moderno** se estructura en torno a **Objetivos de Negocio y Resultados Deseados (Outcomes)**, habitualmente estructurado en tres horizontes temporales flexibles:

1. **Now (Ahora):** Iniciativas en curso dentro de los sprints actuales, con requerimientos altamente refinados y compromiso de entrega a corto plazo.
2. **Next (Próximo):** Iniciativas prioritarias a abordar en las próximas iteraciones; los problemas están comprendidos pero la solución específica aún se está explorando en la vía de Discovery.
3. **Later (Más adelante):** Ideas y oportunidades estratégicas a futuro con baja granularidad y alto nivel de abstracción, sujetas a repriorización según evolucione el mercado.

Módulo 2.2: Requerimientos Ágiles y User Stories

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 1-18) | filmas de clase (03 y 04) | resumen final: páginas (36-50)

Fundamentos de Requerimientos Ágiles: El Manifiesto Ágil y sus Principios

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 1-5) | filmas de clase (03, diapositivas 2-12)

PREGUNTA DE EXAMEN FINAL

¿Por qué son ágiles los requerimientos ágiles?

Son ágiles porque no pretenden congelar una especificación exhaustiva y definitiva al inicio del proyecto. Se gestionan con diferentes niveles de granularidad bajo el concepto de *Just-In-Time*, reconociendo que los requerimientos son emergentes. Su formulación es liviana y negociable, centrada en el valor para el negocio y no en la burocracia documental.

En febrero de 2001, 17 líderes de la industria formularon en Snowbird (Utah) el **Manifiesto por el Desarrollo Ágil de Software**, estructurado en **4 Valores Fundamentales** y **12 Principios**:

Los 4 Valores del Manifiesto Ágil

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 2-3) | filmas de clase (03, diapositivas 6-7) *Aunque reconocemos el valor de los elementos de la derecha, valoramos más los de la izquierda:*

1. **Individuos e interacciones** por sobre procesos y herramientas.
2. **Software funcionando** por sobre documentación extensiva.
3. **Colaboración con el cliente** por sobre negociación contractual.
4. **Respuesta ante el cambio** por sobre seguir un plan preestablecido.

Análisis Conceptual de los 12 Principios del Manifiesto Ágil

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 3-5) | filminas de clase (03, diapositivas 8-12)

1. *Mayor prioridad: satisfacer al cliente mediante la entrega temprana y continua de software con valor.*
2. *Aceptar requerimientos cambiantes, incluso en etapas avanzadas del desarrollo; los procesos ágiles aprovechan el cambio como ventaja competitiva.*
3. *Entregar software funcional frecuentemente, entre dos semanas y dos meses, con preferencia al plazo más corto.*
4. *Los responsables de negocio y los desarrolladores deben trabajar juntos de forma cotidiana durante todo el proyecto.*
5. *Construir proyectos alrededor de individuos motivados, brindándoles el entorno, el apoyo y la confianza necesaria para hacer el trabajo.*
6. *El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.*
7. *El software funcionando es la medida primordial de progreso de un proyecto.*
8. *Los procesos ágiles promueven el desarrollo sostenible; patrocinadores, desarrolladores y usuarios deben mantener un ritmo constante de forma indefinida.*
9. *La atención continua a la excelencia técnica y al buen diseño mejora la agilidad y reduce la deuda técnica.*
10. *La simplicidad (el arte de maximizar la cantidad de trabajo no realizado) es esencial para evitar el desperdicio.*
11. *Las mejores arquitecturas, requerimientos y diseños emergen de equipos autoorganizados.*
12. *A intervalos regulares, el equipo reflexiona sobre cómo ser más eficaz para afinar y ajustar su comportamiento en consecuencia.*

Requerimientos en Ágil y la Superación del BRUF (Big Requirements Up Front)

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 5-7) | filminas de clase (03, diapositivas 13-18)

En el paradigma tradicional, se invertía un esfuerzo masivo al inicio del ciclo de vida para producir la especificación exhaustiva de requerimientos de software (**BRUF**). Los estudios empíricos de la cátedra y del CHAOS Report demuestran que:

- **El 64% de las funcionalidades desarrolladas en proyectos tradicionales casi nunca o nunca son utilizadas por los usuarios finales** (ejemplo clásico: el porcentaje ínfimo de comandos de hojas de cálculo como Microsoft Excel que el usuario común realmente emplea).
- **El 50% de los requerimientos reales son emergentes:** no existen ni pueden conocerse al inicio; aparecen únicamente cuando el usuario comienza a operar el sistema en un entorno de producción real.

- El enfoque ágil invierte la lógica: se identifica un 50% inicial suficiente para concebir el MVP y comenzar a producir valor inmediatamente. El resto de los requerimientos se descubren y refinan de manera *Just-In-Time* a través del feedback del software funcionando.

Jerarquía de Abstracción: Temas, Épicas e Historias de Usuario

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (página 15) | filminas de clase (03, diapositivas 22-25)

El Product Backlog gestiona las necesidades en diferentes escalas de abstracción y tamaño temporal:

Nivel de Abstracción	Definición y Alcance	Horizonte Temporal	Ejemplo de Cátedra
Tema (Theme)	Agrupador estratégico de muy alto nivel que orienta una meta corporativa global.	Varios meses o trimestres de trabajo.	"Modernización integral del ecosistema de cobros y facturación."
Épica (Epic)	Historia de usuario de gran dimensión y complejidad que no cabe en una iteración única. Debe dividirse.	Varios sprints de desarrollo.	"Implementar módulo de pago mediante billeteras virtuales y códigos QR."
Historia de Usuario (User Story)	Unidad mínima de funcionalidad que entrega valor de negocio independiente y cabe en un solo sprint.	Días (dentro de una iteración de 2 a 4 semanas).	"Como cliente registrado, quiero pagar mediante código QR para completar mi compra sin usar efectivo ni tarjeta física."

Anatomía de una User Story y el Modelo de las 3 C's

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 10-13) | filminas de clase (04, diapositivas 2-15)

Una **Historia de Usuario (US)** es un recordatorio de una conversación técnica y funcional entre el negocio y el equipo de desarrollo. Se redacta desde la perspectiva del usuario en el lenguaje del dominio del problema mediante la plantilla estándar:

Como [rol / tipo de usuario], quiero [acción / capacidad del sistema], para [beneficio / valor de negocio esperado].

El Modelo de las 3 C's de Ron Jeffries

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 10-11) | filminas de clase (04, diapositivas 6-8) Toda historia de usuario rigurosa se compone de tres elementos constitutivos:

1. **Card (Tarjeta):** El soporte físico o digital donde se escribe la frase sintética (quién, qué y para qué). Su espacio reducido garantiza que no se convierta en una especificación burocrática cerrada, actuando como una "ficha de invitación a conversar".
2. **Conversation (Conversación):** El diálogo continuo y colaborativo cara a cara entre el Product Owner, los desarrolladores y los testers. En estas reuniones se aclaran dudas, se debaten soluciones arquitectónicas y se exploran alternativas funcionales.
3. **Confirmation (Confirmación):** Los criterios de aceptación y pruebas verificables documentados al dorso de la tarjeta o en el sistema de gestión. Definen de manera inequívoca cuándo la historia está completamente satisfecha y lista para ser validada por el Product Owner.

Criterios de Aceptación y Pruebas con Formato BDD

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 12-14) |
filminas de clase (04, diapositivas 36-42)

Los **Criterios de Aceptación** delimitan las fronteras de una Historia de Usuario. Ayudan a los desarrolladores a saber cuándo dejar de agregar código y permiten derivar los casos de prueba formales.

REGLA DE ORO: QUÉ, NO CÓMO

Los criterios de aceptación deben formular una **intención funcional de negocio (el qué)** y ser totalmente independientes de la solución tecnológica de implementación (el cómo). Detalles específicos de diseño (como si se usa una lista desplegable o una casilla de verificación, o el color de un botón) pertenecen a lineamientos de diseño UI/UX o a pruebas automatizadas de interfaz, no a los criterios de negocio centrales.

Para formalizar la confirmación, se emplea habitualmente el formato de **Desarrollo Guiado por Comportamiento (BDD / Gherkin)**:

- **Dado (Given):** El contexto o estado inicial del sistema antes de la acción.
- **Cuando (When):** La acción o evento desencadenante ejecutado por el usuario.
- **Entonces (Then):** El resultado observable esperado o cambio de estado del sistema.

Criterios de Calidad INVEST para Historias de Usuario

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 14-15) |
filminas de clase (04, diapositivas 21-25)

Formulado por Bill Wake, el acrónimo **INVEST** define las 6 propiedades indispensables que debe cumplir una User Story para estar lista para entrar a un sprint:

- **I - Independent (Independiente):** La historia debe tener el menor acoplamiento posible con otras historias. Debe poder calendarizarse, desarrollarse, probarse y desplegarse en cualquier

orden sin dependencias bloqueantes cruzadas.

- **N - Negotiable (Negociable):** No es un contrato estricto ni una especificación cerrada. El Product Owner y los desarrolladores tienen libertad para negociar el alcance exacto y la solución técnica durante la conversación.
- **V - Valuable (Valiosa):** Debe entregar un valor directo, tangible y explícito para el cliente o usuario final. No se redactan historias de usuario técnicas puras (ej. "Crear tablas de la base de datos" no es una US válida porque no entrega valor por sí sola; debe formar parte de una porción vertical).
- **E - Estimatable (Estimable):** El equipo debe comprender la historia con suficiente claridad técnica y funcional para asignarle un tamaño relativo. Si una historia tiene tanta incertidumbre que no puede estimarse, se transforma temporalmente en un *Spike*.
- **S - Small (Pequeña):** Debe tener el tamaño adecuado para ser planificada, codificada y probada de forma completa dentro de una única iteración (sprint), eliminando el síndrome del 90% terminado.
- **T - Testable (Testable / Comprobable):** Debe ser susceptible de ser verificada mediante pruebas de aceptación automatizadas o manuales. Si no es comprobable, no se puede demostrar si está realmente hecha.

Técnicas de División Vertical de Historias (Story Slicing)

Fuente: filminas de clase (04, diapositivas 43-45) | resumen final: páginas (48-49)

REGLA DE SLICING VERTICAL

Las Historias de Usuario deben dividirse siempre en **rebanadas verticales (*Vertical Slices*)**. Una historia debe atravesar todas las capas arquitectónicas del software: Interfaz de Usuario (UI) + Lógica de Negocio / Servicios + Acceso a Datos / Base de Datos. Dividir horizontalmente por capas técnicas (ej. un sprint para hacer solo bases de datos y otro para hacer pantallas) viola el principio ágil de entregar incrementos funcionales utilizables.

Las estrategias canónicas de división de épicas en historias más pequeñas son:

1. **Por pasos del flujo de trabajo (Workflow Steps):** Implementar primero el flujo básico de inicio a fin (camino feliz) y luego crear historias separadas para pasos intermedios alternativos o avanzados.
2. **Por reglas de negocio:** Separar reglas complejas o validaciones secundarias del comportamiento principal.
3. **Por variaciones de datos o formatos:** Construir primero el soporte para un formato primario (ej. pagos con tarjeta de crédito) y diferir a historias independientes otros métodos (tarjetas de débito, transferencias bancarias).
4. **Por canales o interfaces:** Diferenciar entre la interfaz web estándar y variantes operativas para canales móviles o kioscos de autoservicio.

Spikes de Investigación: Técnicos y Funcionales

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (páginas 16-17) |
filminas de clase (04, diapositivas 16-18)

Un **Spike** es un tipo especial de ítem de trabajo de duración estrictamente acotada (*timeboxed*) diseñado para **quitar riesgo e incertidumbre** técnica o funcional de una historia antes de que el equipo se comprometa a desarrollarla.

- **Spike Técnico:** Se utiliza para investigar la viabilidad en el dominio de la solución tecnológica (ej. evaluar el rendimiento de un nuevo framework, decidir entre comprar un componente comercial o construirlo internamente, o comprobar la latencia de una API de integración externa).
- **Spike Funcional:** Se utiliza para despejar incertidumbres sobre la interacción con el usuario o la lógica del dominio del negocio (ej. crear prototipos rápidos para evaluar el comportamiento del usuario frente a una visualización gráfica compleja).

Lineamientos Críticos para el Uso de Spikes

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (página 17) | filminas de clase (04, diapositiva 18)

- Son la **excepción, no la regla**. Toda historia convive con cierto grado de incertidumbre natural; los spikes se reservan exclusivamente para incógnitas arquitectónicas o funcionales críticas de gran impacto.
- Tienen un tiempo de vida estrictamente acotado (*timeboxed* de pocas horas o días).
- Se planifican en una iteración previa a la iteración donde se construirá la historia resultante, salvo que el spike sea tan pequeño que pueda resolverse y dar paso a la historia en el mismo sprint.

User Story Mapping: Estructuración Visual del Backlog

Fuente: filminas de clase (04, diapositivas 46-55) | resumen final: páginas (49-50)

Técnica formulada por Jeff Patton para evitar la trampa del Product Backlog como una simple lista plana unidimensional donde se pierde el contexto global de la experiencia del usuario.

- **Eje Horizontal (Tiempo y Flujo del Usuario):** Representa la secuencia cronológica de actividades del usuario (denominada **Backbone** o Columna Vertebral).
- **Eje Vertical (Profundidad y Prioridad):** Las historias de usuario se disponen debajo de cada paso del backbone de arriba hacia abajo, según su valor de negocio y necesidad.
- **Walking Skeleton (Esqueleto Caminante):** La primera fila horizontal completa que atraviesa todo el mapa. Constituye la versión funcional mínima de extremo a extremo que permite validar la arquitectura y el flujo esencial de negocio. Las filas horizontales subsiguientes definen los lanzamientos o *releases* sucesivos (Release 1 = MVP, Release 2 = MMP, etc.).

Usuarios Representantes (Proxies) y Gestión de Stakeholders

Fuente: apunte de clase: Requerimientos Ágiles y User Stories (página 12) | filminas de clase (03, diapositiva 28)

En el mundo ideal, el Product Owner interactúa de forma directa y cotidiana con los usuarios finales auténticos. Sin embargo, en grandes organizaciones corporativas, productos de consumo masivo o sistemas con regulaciones complejas, se recurre a la figura de los **Usuarios Representantes (Proxies)**: gerentes de usuarios, analistas de marketing, ejecutivos de ventas, expertos del dominio (SMEs) o personal de mesa de ayuda y soporte técnico.

RIESGO ASOCIADO AL USO DE PROXIES

Aunque los proxies son necesarios cuando el acceso a los usuarios finales es inviable, introducen un sesgo cognitivo inevitable: tienden a transmitir lo que ellos creen que el usuario necesita en lugar del comportamiento real del mercado. Se recomienda validar siempre las definiciones de los proxies mediante pruebas empíricas de usabilidad y telemetría de producto.

Módulo 2.3: Estimaciones en Ambientes Ágiles

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 1-9) | filminas de clase (04 y 07) | resumen final: páginas (51-60)

Naturaleza de la Estimación en Software: Incertidumbre, Probabilidad y Cono de Incertidumbre

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 1-3) | filminas de clase (07, diapositivas 2-8)

Cualquier proceso de estimación en ingeniería de software opera inevitablemente en un universo de **incertidumbre**, con una **probabilidad asociada**. Lo contrario a la estimación no es una mejor estimación, sino la certeza absoluta, y la certeza solo se alcanza cuando el software está terminado y desplegado.

EL CONO DE LA INCERTIDUMBRE (BARRY BOEHM / STEVE MCCONNELL)

Al inicio del ciclo de vida de un proyecto, la incertidumbre sobre el tiempo y costo real es máxima, con un factor de error de **hasta 4x (un rango de 0.25x a 4.0x)**. A medida que avanza el proyecto y se toman decisiones de arquitectura y diseño, el cono se estrecha.

La respuesta ágil: El agilismo postula que no se puede planificar exhaustivamente al inicio para eliminar la incertidumbre. La incertidumbre solo se elimina **iterando para descubrirla**: construyendo software funcional que genere retroalimentación real.

Matriz de Desambiguación: Objetivo vs. Estimación vs. Compromiso

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 2-3) | resumen final: página (51)

Uno de los errores más graves en la administración de proyectos es confundir estos tres conceptos fundamentales:

Concepto	Definición y Naturaleza	¿Quién lo Define?	¿Es Negociable?
Objetivo (Target)	Una meta de negocio deseada por la organización (ej. lanzar la plataforma antes de Navidad). Es subjetivo.	El Negocio / Clientes / Product Owner.	Fijo o deseado por el negocio.
Estimación (Estimate)	Una predicción analítica e imparcial del tamaño o esfuerzo requerido, basada en parámetros observados.	El equipo técnico que realizará el trabajo.	NUNCA SE NEGOCIA. Es un cálculo probabilístico.
Compromiso (Commitment)	Una promesa de entrega formal acordada asumiendo riesgos específicos tras evaluar la estimación frente al objetivo.	El equipo junto al Product Owner y stakeholders.	SÍ se negocia ajustando el alcance.

AXIOMA DE CÁTEDRA

Las estimaciones nunca se negocian. Lo que se negocia son los compromisos. Negociar una estimación para forzarla a coincidir con un objetivo de negocio arbitrario no acelera el desarrollo; destruye las bases de ingeniería, asegura soluciones rápidas y defectuosas y traslada los errores a la fase de producción, donde corregirlos resulta astronómicamente más caro.

Estimaciones Relativas vs. Absolutas: Psicología y Story Points

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 3-5) | filmas de clase (04, diapositivas 4-10)

La psicología cognitiva demuestra de forma concluyente que los seres humanos somos naturalmente deficientes al estimar magnitudes absolutas (ej. adivinar si una tarea tardará 37 horas y media), pero somos notablemente precisos estimando por **comparación relativa** (determinar con rapidez que la tarea B es el doble de compleja que la tarea A).

En el paradigma ágil, se abandona la estimación en horas lineales individuales y se adopta una medida abstracta de tamaño relativo: el **Story Point (Punto de Historia)**.

LAS TRES DIMENSIONES DEL STORY POINT

El Story Point es una unidad combinada que sintetiza simultáneamente tres factores intrínsecos:

1. **Complejidad Técnica:** Qué tan intrincada, desafiante o no lineal es la lógica y la arquitectura a implementar.
2. **Esfuerzo Requerido:** El volumen total de trabajo a realizar (horas-hombre relativas para codificar, documentar y probar).
3. **Incertidumbre y Riesgo:** Lo desconocido, la falta de experiencia con una librería o dependencias externas no controladas.

Escalas de Estimación y la Serie de Fibonacci Modificada

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 4-6) | filminas de clase (04, diapositivas 11-15)

Dado que la complejidad del software y la incertidumbre crecen de forma exponencial con el tamaño, las escalas lineales (como 1, 2, 3, 4, 5...) resultan inapropiadas para estimar ítems grandes.

La cátedra adopta la **Serie de Fibonacci Modificada** popularizada por Mike Cohn:

Escala: 0 | 0.5 | 1 | 2 | 3 | 5 | 8 | 13 | 20 | 40 | 100 | ? | 🍷

- **Por qué se redondea a 20, 40 y 100:** En la serie matemática pura continuarían 21, 34, 55 y 89. Se reemplazan deliberadamente por valores redondeados para **evitar una falsa sensación de precisión**. Debatir si una historia tiene 20 o 21 puntos carece de sentido cognitivo y desperdicia tiempo valioso.
- **Valores Especiales:**
 - 0 : Para requerimientos triviales o de configuración que casi no insumen esfuerzo.
 - 0.5 : Para tareas sumamente pequeñas que son más sencillas que la historia base.
 - ? : Expresa total desconocimiento; la historia no puede estimarse y debe convertirse en un *Spike*.
 - 🍷 : Carta de descanso para cuando el equipo experimenta fatiga cognitiva durante la sesión.
- **T-Shirt Sizing (Talles de Remera - XS, S, M, L, XL, XXL):** Se emplea como técnica de estimación rápida y de grano grueso para ordenar los ítems en las fases iniciales del Product Backlog antes de que pasen a refinamiento detallado.

Dinámica del Planning Poker (Scrum Poker) Paso a Paso

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 5-7) | filminas de clase (04, diapositivas 16-25)

Técnica gamificada y colaborativa que garantiza estimaciones colectivas y elimina sesgos psicológicos:

1. **Presentación de la Historia:** El Product Owner lee la Historia de Usuario y explica los criterios de aceptación y el valor de negocio esperado.
2. **Aclaración Técnica:** El equipo realiza preguntas breves para despejar dudas sobre dependencias o arquitectura.
3. **Votación a Ciegas (Secreta):** Cada integrante selecciona una carta con un valor de la serie de Fibonacci modificada y la coloca boca abajo sobre la mesa (o en la herramienta digital). Nadie puede expresar números en voz alta antes de votar. **Esto destruye el "sesgo de anclaje" y la influencia por autoridad técnica.**
4. **Revelación Simultánea:** A la cuenta de tres, todos los integrantes muestran su carta al mismo tiempo.
5. **Debate de los Extremos:** Si hay consenso, se anota el valor. Si hay discrepancia notable (ej. votos de 2 y 13):
 - **El voto más bajo:** Explica por qué considera que la tarea es sencilla (quizás conoce una librería o un componente preexistente que resuelve el problema fácilmente).
 - **El voto más alto:** Explica qué riesgos ocultos, complejidades arquitectónicas o casos de borde identificó que los demás pasaron por alto.
6. **Re-votación y Consenso:** Tras un debate breve (2 a 3 minutos), el equipo vuelve a votar en secreto. Por lo general, en la segunda ronda se alcanza el acuerdo de manera natural.

Velocidad del Equipo, Capacidad y la Ecuación Central Ágil

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 5, 8-9) | filminas de clase (04, 08)

DEFINICIÓN DE VELOCIDAD DE CÁTEDRA

La **Velocidad** es la cantidad total de Story Points correspondientes a las Historias de Usuario que han sido **completamente terminadas y aceptadas formalmente por el Product Owner** al concluir el Sprint.

La velocidad NO se estima; se CALCULA de manera empírica. La gestión ágil es binaria: una historia con 99% de avance no aporta ningún punto de velocidad hasta que cumpla la Definición de Hecho (DoD) y sea aceptada.

Cálculo de la Capacidad del Equipo

Fuente: apunte de clase: Estimaciones en ambientes ágiles (página 8) | filminas de clase (04, diapositiva 28) La capacidad mide el tiempo operativo real que el equipo tendrá disponible para trabajar en una iteración:

$$\text{Capacidad Bruta} = \text{Horas Disponibles por Día (WH)} \times \text{Días Laborables de la Iteración (DA)}$$

La capacidad neta o factor de foco se calcula deduciendo de la capacidad bruta los tiempos destinados a reuniones de Scrum, soporte operativo, capacitaciones y margen de imprevistos (generalmente un factor de dedicación del 60% al 80%).

La Ecuación Central de la Planificación Ágil

Fuente: apunte de clase: Estimaciones en ambientes ágiles (página 8) | filminas de clase (04, diapositiva 29) En los entornos ágiles, nunca se estima el tiempo de forma directa. Se estima el tamaño del trabajo en Story Points, se observa la velocidad empírica del equipo y se calcula matemáticamente el cronograma:

$$\text{Duración del Proyecto (Sprints)} = \frac{\text{Tamaño Total del Backlog (Story Points)}}{\text{Velocidad Empírica Promedio (Story Points / Sprint)}}$$

Los Cuatro Fundamentos que Hacen Ágiles a las Estimaciones

Fuente: apunte de clase: Estimaciones en ambientes ágiles (páginas 8-9) | resumen final: páginas (54-55)

1. **Son Relativas y no Absolutas:** Desechan las horas individuales lineales; estiman por comparación de magnitudes frente a una historia base canónica (Story Points).
2. **Son Just-In-Time y combaten el Cono de la Incertidumbre:** No se estima todo al inicio cuando menos información se tiene. Se estima en detalle únicamente el trabajo de la próxima iteración, permitiendo reestimar si se descubre nueva complejidad.
3. **Son Colectivas y Descentralizadas:** Estima de forma democrática el equipo que ejecutará el trabajo (Planning Poker), eliminando la imposición autocrática de un líder y unificando el criterio técnico.
4. **Se basan en el Empirismo:** La duración real es una consecuencia matemática calculada a través de la velocidad histórica demostrada por el equipo en producción.

Módulo 2.4: Framework Scrum (Guía 2020 y Expansión 2026)

Fuente: filminas de clase (08, diapositivas 1-65) | Guía Scrum 2020 | Guía de Expansión Scrum 2026 | Apuntes_SCRUM_2020_y_Expansion_2026

Fundamentos de Scrum: Definición, Pilares Empíricos y Valores

Fuente: filminas de clase (08, diapositivas 2-10) | Guía Scrum 2020 | Apuntes_SCRUM_2020_y_Expansion_2026

Scrum es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos. Creado por Ken Schwaber y Jeff Sutherland, se sustenta sobre el **empirismo** y el **pensamiento Lean** (reducir el desperdicio y enfocarse en lo esencial).

Los Cinco Valores de Scrum

Fuente: filminas de clase (08, diapositivas 7-10) | Guía Scrum 2020 El empleo exitoso de Scrum depende de que las personas convivan con cinco valores éticos:

1. **Compromiso (Commitment):** El equipo se compromete a alcanzar sus metas y a apoyarse mutuamente.
2. **Foco (Focus):** El foco principal del sprint es el trabajo del Sprint Backlog para avanzar hacia el Sprint Goal.
3. **Franqueza / Apertura (Openness):** El equipo y sus interesados acuerdan ser transparentes acerca del trabajo y los desafíos.
4. **Respeto (Respect):** Los miembros del equipo se respetan entre sí como personas capaces e independientes.
5. **Coraje (Courage):** El equipo tiene el coraje de hacer lo correcto y abordar problemas difíciles.

Estructura del Scrum Team y Roles / Accountabilities

Fuente: filminas de clase (08, diapositivas 11-18) | Guía Scrum 2020 | Apuntes_SCRUM_2020_y_Expansion_2026

En la Guía Scrum 2020 se eliminó el concepto de "roles" y jerarquías internas, reemplazándolo por **Responsabilidades (Accountabilities)** dentro de una única unidad cohesiva de profesionales: el **Scrum Team**. No hay subequipos ni jerarquías; es un equipo multifuncional y autogestionado de tamaño típicamente pequeño (10 o menos personas).

Product Owner (PO)

Responsabilidad: Maximizar el valor del producto resultante del trabajo del Scrum Team.

Actividades Clave: Es el único dueño y responsable de la gestión efectiva del Product Backlog (crear y comunicar el Product Goal, redactar PBIs, ordenarlos por valor y asegurar que el backlog sea transparente y comprensible). Representa las necesidades del negocio.

Scrum Master (SM)

Responsabilidad: Establecer Scrum según se define en la Guía y velar por la efectividad del equipo.

Actividades Clave: Líder que sirve al equipo (Líder Servidor). Facilita eventos, remueve impedimentos organizacionales que bloquean a los desarrolladores y apoya al Product Owner en técnicas de gestión del backlog.

Developers (Desarrolladores)

Responsabilidad: Crear cualquier aspecto de un Incremento utilizable y terminado en cada Sprint.

Actividades Clave: Crear el plan para el Sprint (Sprint Backlog), inculcar calidad adhiriéndose a la Definition of Done, adaptar su plan cada día hacia el Sprint Goal y responsabilizarse mutuamente como profesionales técnicos.

La Expansión Scrum 2026: SGEP y Nuevas Dinámicas de Trabajo

Fuente: filminas de clase (08, diapositivas 19-28) | Guía de Expansión Scrum 2026 | Apuntes_SCRUM_2020_y_Expansion_2026

La evolución contemporánea de la cátedra incorpora las dimensiones de la **Guía de Expansión Scrum 2026 (SGEP)**, adaptando el marco a entornos de inteligencia artificial y desarrollo acelerado:

- **Integración de Agentes de IA en el Flujo de Trabajo:** La IA actúa como un copiloto de desarrollo y análisis de datos en tareas rutinarias (generación de scaffolding de código, redacción preliminar de pruebas unitarias y monitoreo automatizado de logs).
- **El Rol del Supporter (Soporte Continuo):** Se formaliza una figura o responsabilidad encargada de la salud estructural a largo plazo: resolución proactiva de deuda técnica acumulada, parcheo de librerías, actualización de pipelines de CI/CD y atención de incidentes menores de producción sin interrumpir el foco del Scrum Team principal.
- **Principio Human-in-the-Loop (HITL):** Enfatiza que ninguna decisión arquitectónica crítica, compromiso ético o aprobación de criterios de aceptación de negocio puede delegarse ciegamente a algoritmos o IA. La responsabilidad final (*accountability*) sigue residiendo enteramente en los profesionales humanos.
- **Redefinición de la Definition of Done orientada a Outcome:** La DoD ya no se detiene en certificar que el código compile, tenga pruebas y se despliegue (output). En la visión 2026, la DoD incorpora métricas de observabilidad en producción y verificación de impacto funcional real (outcome) mediante telemetría de uso.

Los Eventos de Scrum: Entradas, Time-Box y Definición Completa

Fuente: filminas de clase (08, diapositivas 29-45) | Guía Scrum 2020 | Guía de Expansión Scrum 2026

A continuación se detalla cada ceremonia oficial de Scrum con sus entradas, duración máxima y definición exhaustiva para examen:

1. El Sprint

Fuente: filminas de clase (08, diapositivas 29-31) | Guía Scrum 2020

- **Entradas:** Visión de Producto, Product Backlog refinado, capacidad proyectada del equipo, estado de la arquitectura tecnológica y aprendizajes de la retrospectiva previa.
- **Time-Box:** Ciclo fijo de **1 a 4 semanas** (un mes calendario o menos). Su duración debe ser constante a lo largo de todo el ciclo de vida del producto para generar un ritmo de trabajo predecible y sostenible.
- **Definición para Examen:** El Sprint es el evento contenedor de todos los demás eventos de Scrum. Es un bloque de tiempo de duración fija durante el cual se transforma una idea en un Incremento de producto utilizable y potencialmente desplegable que aporta valor de negocio.

Durante el Sprint no se realizan cambios que pongan en peligro el Objetivo del Sprint (Sprint Goal), no se disminuye la calidad y el alcance se puede clarificar y renegociar con el Product Owner a medida que se aprende más.

2. Sprint Planning (Planificación del Sprint)

Fuente: filminas de clase (08, diapositivas 32-35) | Guía Scrum 2020

- **Entradas:** Product Backlog priorizado y refinado, el último Incremento de producto entregado, la capacidad disponible del equipo para la iteración y el desempeño histórico (velocidad empírica).
- **Time-Box:** Máximo de **8 horas para un Sprint de un mes** (proporcionalmente menor para sprints más cortos, ej. 4 horas para sprints de dos semanas).
- **Definición para Examen:** Evento colaborativo que da inicio al Sprint donde todo el Scrum Team define qué trabajo se realizará y cómo se ejecutará. Se estructura respondiendo a tres temas cardinales:
 1. *¿Por qué es valioso este Sprint?* El Product Owner propone cómo el producto podría incrementar su valor y el equipo define de manera consensuada el **Sprint Goal (Objetivo del Sprint)**.
 2. *¿Qué se puede hacer en este Sprint?* Los Developers seleccionan los ítems del Product Backlog que se comprometen a completar según su capacidad y velocidad empírica.
 3. *¿Cómo se realizará el trabajo elegido?* Los Developers descomponen los ítems seleccionados en tareas técnicas de granularidad fina (generalmente de un día o menos de esfuerzo) conformando el **Sprint Backlog**.

3. Daily Scrum (Scrum Diario)

Fuente: filminas de clase (08, diapositivas 36-38) | Guía Scrum 2020

- **Entradas:** El Sprint Goal, el Sprint Backlog actual, el estado del tablero de tareas y los impedimentos identificados en la jornada previa.
- **Time-Box:** Estricto límite de **15 minutos diarios**, celebrado a la misma hora y en el mismo lugar físico o virtual durante todos los días hábiles del Sprint.
- **Definición para Examen:** Reunión técnica diaria de inspección y sincronización exclusiva para los Developers. Su propósito es inspeccionar el avance hacia el Sprint Goal y adaptar el Sprint Backlog según sea necesario, ajustando el plan de trabajo para las siguientes 24 horas. Aunque no es obligatorio el formato rígido de tres preguntas, tradicionalmente evalúa qué se logró ayer para acercarse al objetivo, qué se hará hoy para contribuir al objetivo y qué impedimentos están bloqueando el avance. No es una reunión de reporte de estado para el Scrum Master o jefes, sino un evento interno de autoorganización técnica.

4. Sprint Review (Revisión del Sprint)

Fuente: filminas de clase (08, diapositivas 39-41) | Guía Scrum 2020

- **Entradas:** El Incremento de producto que cumple rigurosamente con la Definition of Done, el Product Goal actual y la información sobre cambios en el entorno de mercado o presupuesto.
- **Time-Box:** Máximo de **4 horas para un Sprint de un mes** (generalmente 2 horas para sprints de dos semanas).
- **Definición para Examen:** Evento informal de inspección y adaptación al cierre del Sprint donde el Scrum Team y los interesados clave (*stakeholders*) revisan qué se logró durante la iteración. Los Developers demuestran operativamente el software funcionando que cumple la DoD (no se muestran diapositivas ni maquetas estáticas). Se analiza el impacto del nuevo incremento en el negocio y se debaten colaborativamente los próximos pasos, adaptando el Product Backlog para reflejar las nuevas oportunidades de mercado descubiertas.

5. Sprint Retrospective (Retrospectiva del Sprint)

Fuente: filminas de clase (08, diapositivas 42-44) | Guía Scrum 2020

- **Entradas:** Observaciones empíricas del equipo sobre personas, relaciones interpersonales, procesos, herramientas y la Definition of Done utilizada durante el Sprint concluido.
- **Time-Box:** Máximo de **3 horas para un Sprint de un mes** (generalmente 1.5 horas para sprints de dos semanas).
- **Definición para Examen:** Evento interno y confidencial donde el Scrum Team inspecciona cómo se desarrolló el último Sprint en relación con las personas, las interacciones, los procesos técnicos y las herramientas de trabajo. Su fin primordial es identificar qué funcionó bien, qué problemas surgieron y cómo fueron abordados, planificando mejoras concretas para incrementar la calidad y eficacia en el siguiente Sprint. Es el motor fundamental de la mejora continua ágil (*Kaizen*).

Actividad Clave: Refinamiento del Product Backlog (Backlog Refinement)

Fuente: filminas de clase (08, diapositiva 45) | Guía Scrum 2020

- **Definición:** Actividad continua (no es un evento oficial formal con timebox cerrado, pero sí una práctica indispensable) en la que el Product Owner y los Developers añaden detalle, estimaciones analíticas y orden a los ítems del Product Backlog. Consume habitualmente hasta un **10% de la capacidad total del equipo** durante el sprint y garantiza que los ítems alcancen el estado de "Listo" (DoR) para el próximo Sprint Planning.

Los Tres Artefactos Oficiales y sus Compromisos

Fuente: filminas de clase (08, diapositivas 46-55) | Guía Scrum 2020 | Apuntes_SCRUM_2020_y_Expansion_2026

En Scrum 2020, cada uno de los 3 artefactos oficiales contiene un **Compromiso formal** destinado a reforzar el empirismo y medir el progreso:

1. Product Backlog

Definición: Lista ordenada y emergente de todo lo que se sabe que es necesario en el producto.

Compromiso Asociado: El Product Goal (Objetivo del Producto).

Describe un estado futuro del producto que sirve como objetivo a largo plazo para el Scrum Team. El equipo debe cumplir o abandonar un objetivo antes de asumir el siguiente.

2. Sprint Backlog

Definición: Conjunto de ítems del Product Backlog seleccionados para el Sprint, junto con el plan ejecutable de tareas para entregarlos.

Compromiso Asociado: El Sprint Goal (Objetivo del Sprint).

Es la única meta del Sprint elegida durante el Sprint Planning. Proporciona coherencia, foco y dirección, permitiendo a los desarrolladores flexibilidad en el alcance de las tareas mientras no se vulnere el objetivo.

3. Increment (Incremento)

Definición: Peldaño concreto hacia el Product Goal. Es la suma acumulada de todos los ítems terminados durante el sprint actual más el valor de los sprints anteriores.

Compromiso Asociado: La Definition of Done (Definición de Terminado).

Descripción formal del estado del Incremento cuando satisface las medidas de calidad requeridas para el producto. Un ítem no puede presentarse en la Review si no cumple al 100% la DoD.

Planificación de Release y Métricas Ágiles en Scrum

Fuente: filminas de clase (08, diapositivas 56-65) |
Apuntes_SCRUM_2020_y_Expansion_2026

Para proyectar entregas a mediano y largo plazo sin recurrir a cronogramas predictivos ficticios, Scrum emplea herramientas visuales empíricas:

- **Gráfico de Burndown (Trabajo Restante):** Grafica en el eje vertical los Story Points restantes y en el eje horizontal el tiempo (días del sprint o sprints del release). Permite detectar desvíos tempranos:
 - Si la línea real está por encima de la ideal: el equipo está retrasado o hubo ampliación no planificada del alcance.
 - Si está por debajo: el equipo terminará antes de lo previsto o se sobreestimó el esfuerzo.
- **Gráfico de Burnup (Trabajo Completado y Alcance Total):** Muestra dos líneas simultáneas: el total de trabajo completado y el alcance total del proyecto. Permite distinguir de forma inmediata si un retraso obedece a un rendimiento menor del equipo o a que el Product Owner agregó nuevos requerimientos al Product Backlog.
- **Métricas de Flujo Lean:**
 - **Lead Time (Tiempo de Entrega):** Tiempo transcurrido desde que surge la necesidad y se registra en el backlog hasta que está en producción entregando valor.
 - **Cycle Time (Tiempo de Ciclo):** Tiempo transcurrido desde que el equipo comienza a codificar activamente una historia hasta que cumple la DoD.
 - **Throughput (Rendimiento):** Cantidad de ítems completados por unidad de tiempo.

- **Defect Escape Rate:** Porcentaje de defectos que escapan de las pruebas del sprint y son descubiertos por usuarios reales en producción.

Unidad 3: Software Configuration Management (Gestión de Configuración de Software)

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 11-24) | filminas de clase (02) | resumen final: páginas (19-28)

Definición y Alcance de SCM: La Disciplina Paraguas

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 11-12) | filminas de clase (02, diapositiva 3)

CONCEPTO CENTRAL

¿Qué es la configuración del software?

Es el conjunto estructurado de Ítems de Configuración (ICs), cada uno con su versión específica y aprobada, en un momento determinado del tiempo.

¿Qué es la Gestión de Configuración de Software (SCM)?

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 11) | filminas de clase (02, diapositiva 3) Es una disciplina de soporte, transversal a todo el proyecto y relevante para el producto a lo largo de todo su ciclo de vida (surgida para enfrentar la crisis y el caos en el desarrollo de software), que aplica dirección y monitoreo administrativo y técnico para:

1. **Identificar y documentar** las características técnicas y funcionales de los ítems de configuración.
2. **Controlar sistemáticamente los cambios** sobre esas características identificadas y documentadas.
3. **Registrar e informar** el estado de los cambios y la evolución del sistema.
4. **Verificar y auditar** la correspondencia con los requerimientos aprobados (garantizando la trazabilidad).

LA DISCIPLINA "PARAGUAS"

Se dice formalmente que SCM es una actividad o disciplina "**paraguas**" porque actúa como una estructura protectora y base transversal indispensable: debe funcionar correctamente para que el resto de las disciplinas de ingeniería (requerimientos, análisis, arquitectura, codificación y pruebas) puedan operar de manera íntegra y coordinada.

Definición Simplificada de Cátedra: Es la disciplina de la ingeniería de software que administra sistemáticamente los Ítems de Configuración (ICs) para mantener la **integridad** del software a lo largo de todo su ciclo de vida.

¿Cuándo un Producto de Software es Íntegro?

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 12) | filminas de clase (02, diapositiva 4) Un producto de software alcanza y preserva su integridad cuando satisface concurrentemente cuatro condiciones cardinales:

- Satisface plenamente los requerimientos del cliente.
- Asegura la **trazabilidad bidireccional** de todos sus ítems de configuración.
- Satisface rigurosamente los requerimientos no funcionales (rendimiento, seguridad, escalabilidad, usabilidad).
- Cumple estrictamente con las expectativas presupuestarias y de costo acordadas.

¿Por qué Deberíamos Gestionar la Configuración del Software?

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 13) | filminas de clase (02, diapositiva 5) Sin una gestión rigurosa de configuración, los proyectos caen en el desorden operativo. SCM resuelve patologías comunes de la industria:

- **Pérdida de componentes o cambios:** Situaciones donde la versión sobre la que se trabaja no es la última aprobada, sobrescribiendo avances previos.
- **Superposición de cambios:** Dos o más programadores modifican el mismo archivo simultáneamente sin mecanismos de concurrencia y control de conflictos.
- **Cambios no validados:** Incorporación de modificaciones al código que no han sido autorizadas ni evaluadas en su impacto sistémico.
- **Reintroducción de errores (Bugs Resucitados):** Defectos subsanados en versiones previas que vuelven a aparecer en el software por integrar versiones desactualizadas de librerías o módulos.
- **Doble mantenimiento:** Obligación de corregir el mismo defecto en múltiples ramas o versiones de manera aislada y desincronizada.

Conceptos Nucleares: Ítem de Configuración (IC), Versiones, Revisiones y Variantes

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 13-15) | filminas de clase (02, diapositivas 4-8)

Ítem de Configuración (IC / SCI)

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 13) | filminas de clase (02, diapositivas 6-7) Es una pieza elemental de información o artefacto de software formalmente identificado y registrado, que se coloca bajo el control de la disciplina de SCM para permitir al equipo conocer su estado, versión y evolución histórica a lo largo del tiempo.

- **Regla de Identificación:** *"Todo lo que entra a SCM debe colocarse en un repositorio bajo una identificación unívoca con un nombre único y estandarizado."*
- **Granularidad del IC:** La granularidad (tamaño del elemento) debe ser equilibrada. Una granularidad excesivamente fina (ej. registrar cada método individual como un IC) genera una burocracia inmanejable; una granularidad demasiado gruesa (ej. considerar todo el sistema como un único IC) impide el control fino de cambios.

Versión, Revisión y Variante

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 14-15) | filminas de clase (02, diapositivas 7-8)

- **Versión:** Es una instancia concreta de un ítem de configuración o de una configuración completa en un momento específico del tiempo. Representa una evolución en el estado del artefacto.
- **Revisión:** Es una versión que sucede a otra en el tiempo para corregir un defecto detectado o realizar una optimización menor. Representa una progresión lineal en la misma línea de desarrollo (ej. Versión 1.0.1 que corrige un bug de la 1.0.0).
- **Variante:** Es una versión que coexiste simultáneamente con otra en el tiempo, pero que implementa un comportamiento o característica diferencial para satisfacer necesidades específicas de hardware, sistema operativo o clientes:
 - *Variante de Ítem de Configuración:* Un módulo adaptado para correr bajo entorno Linux y otra variante idéntica en funcionalidad para correr bajo Windows.
 - *Variante de Producto:* Dos ediciones comerciales del mismo software que coexisten en el mercado (ej. Edición Estándar vs. Edición Enterprise con funcionalidades avanzadas).

Repositorio: Centralizado vs. Distribuido

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 15) | filminas de clase (02, diapositiva 9) El repositorio es la base de datos especializada donde se

almacenan todos los ítems de configuración, junto con su historial completo de versiones, metadatos y relaciones de trazabilidad:

- **Repositorios Centralizados (ej. SVN):** Existe un único servidor central que contiene la base de datos de versiones. Los desarrolladores solo descargan una copia de trabajo local (*working copy*); si el servidor falla, se pierde la capacidad de versionar.
- **Repositorios Distribuidos (ej. Git):** Cada desarrollador posee una copia completa y espejo del repositorio histórico en su máquina local. Permite trabajar desconectado, ramificar con costo nulo y ofrece redundancia total.

Ramas (Branching) y Fusión (Merging)

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 16) | filminas de clase (02, diapositiva 10)

- **Creación de Ramas (Branching):** Permite aislar líneas de trabajo independientes del tronco principal (*main/trunk*) para desarrollar funcionalidades experimentales, resolver incidentes urgentes (*hotfixes*) o trabajar en paralelo sin desestabilizar la versión estable.
- **Integración de Ramas (Merging):** Operación técnica que toma los cambios validados en una rama secundaria y los incorpora a la rama principal. Si se presentan conflictos concurrentes (dos desarrolladores modificaron las mismas líneas), estos deben resolverse manualmente antes de consolidar la integración.

Líneas Base (Baselines) y Momentos Críticos del Ciclo de Vida

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 16-17) | filminas de clase (02, diapositivas 9-13)

DEFINICIÓN FORMAL DE LÍNEA BASE (BASELINE)

Una **Línea Base** es una especificación o producto de software que ha sido formalmente revisado y acordado por los interesados, que sirve como base oficial para el desarrollo posterior y que **solo puede modificarse a través de un procedimiento formal de control de cambios**.

Actúan como puntos de referencia estables: permiten retroceder en el tiempo y reproducir de forma determinista el entorno operativo y el código exacto del sistema en un hito histórico determinado. En las herramientas modernas de versionado, se materializan mediante **etiquetas inmutables (Tags)**.

Tipos de Líneas Base en el Ciclo de Vida

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 17) | filminas de clase (02, diapositivas 12-13)

1. **Línea Base Funcional:** Establecida tras la aprobación formal de la Especificación de Requerimientos de Software (ERS). Congela qué debe hacer el producto antes de acometer la arquitectura profunda.
2. **Línea Base Asignada (de Diseño):** Congela la arquitectura global del sistema, la distribución de módulos, las interfaces y el diseño de la base de datos tras las revisiones técnicas de diseño.
3. **Línea Base de Desarrollo:** Marca hitos operativos internos donde un conjunto de componentes de software ha superado las pruebas unitarias y de integración preliminares.
4. **Línea Base del Producto (de Lanzamiento / Release):** Congela el software completamente probado, empaquetado y documentado que supera las pruebas de aceptación y se despliega en el entorno productivo real del cliente.

Las Cuatro Actividades Fundamentales de SCM

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 17-21) | filminas de clase (02, diapositivas 14-22)

La disciplina de SCM se articula en torno a cuatro procesos cardinales continuos:

1. Identificación

Definir qué artefactos serán ICs, convenciones de nombrado unívoco, esquemas de versionado y estructura del repositorio.

2. Control de Cambios

Procedimiento formal para evaluar, autorizar, coordinar y verificar cualquier cambio sobre ICs en línea base.

3. Auditorías de Configuración

Verificación formal física (PCA) y funcional (FCA) para asegurar la integridad técnica y documental de las entregas.

4. Registro e Informe de Estado

Status Accounting: registrar quién, qué, cuándo y por qué se modificó cada IC, reportando métricas de evolución.

Clasificación de los Ítems de Configuración por Ciclo de Vida

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 18) | filminas de clase (02, diapositivas 15-16) La cátedra clasifica rigurosamente los ICs según su supervivencia temporal:

- **ICs de Producto:** Tienen una vida útil prolongada que trasciende la duración del proyecto. Entran en mantenimiento y sobreviven al equipo original: el código fuente, la ERS, los modelos de base de datos, manuales de usuario y suites de pruebas de regresión.

- **ICs de Proyecto:** Mueren con la finalización del proyecto. Sirven para gobernar el esfuerzo temporal: el Plan de Proyecto, la Matriz de Gestión de Riesgos y el cronograma.
 - **ICs de Iteración:** Mueren al concluir la iteración o sprint específico: el Sprint Backlog de la iteración, las minutas de reuniones diarias y el reporte de defectos de esa entrega particular.
-

El Comité de Control de Cambios (CCB) y el Flujo Formal de Solicitudes de Cambio (CR)

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 19-20) | filminas de clase (02, diapositivas 23-28)

El **Comité de Control de Cambios (Change Control Board - CCB)** es la autoridad formal colegiada responsable de evaluar, autorizar o rechazar cualquier modificación solicitada sobre los Ítems de Configuración que ya forman parte de una línea base formal. Está integrado por representantes del cliente, líderes técnicos de desarrollo, especialistas de calidad (QA) y el líder del proyecto.

El Flujo Sistemático de Control de Cambios

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 19) | filminas de clase (02, diapositivas 24-26)

1. **Emisión de la Solicitud de Cambio (CR / RFC):** El interesado (cliente, usuario o desarrollador) documenta formalmente la necesidad de cambio describiendo el motivo y la justificación de negocio.
 2. **Análisis de Impacto Técnico y Económico:** El equipo evalúa qué módulos de código, documentos y casos de prueba se verán afectados, cuantificando el esfuerzo en horas, el desvío en el cronograma y el costo financiero.
 3. **Decisión del CCB:** El comité revisa el informe de impacto y toma una resolución formal:
 - **Aprobar:** Se emite una Orden de Modificación formal y se asignan recursos.
 - **Rechazar:** Se notifica al solicitante documentando la justificación técnica o presupuestaria.
 - **Postergar:** Se difiere la evaluación para una versión o release posterior.
 4. **Implementación y Modificación:** El desarrollador extrae el IC de la línea base (*checkout* o creación de rama), ejecuta la modificación y realiza pruebas unitarias.
 5. **Verificación y Nueva Línea Base:** Aseguramiento de la Calidad verifica que el cambio no haya introducido efectos colaterales (pruebas de regresión) y el IC modificado se incorpora consolidando una nueva línea base (*commit / tag*).
-

Auditorías de Configuración: FCA (Funcional) y PCA (Física)

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 20-21) | filminas de clase (02, diapositivas 29-32)

Las auditorías de configuración garantizan que el software a entregar cumpla con los estándares comprometidos antes de su despliegue en producción:

Auditoría Funcional de Configuración (FCA)

Objetivo: Verificación Funcional.

Comprueba formalmente que cada ítem de configuración haya alcanzado las características funcionales y de rendimiento especificadas en los requerimientos aprobados (ERS). Examina los resultados de los casos de prueba y verifica que el sistema se comporte exactamente como se definió. Responde a la pregunta: *¿El software hace todo lo que se especificó que debía hacer?*

Auditoría Física de Configuración (PCA)

Objetivo: Integridad Estructural y Existencia.

Comprueba que todos los componentes físicos necesarios para compilar, construir, empaquetar y desplegar el software estén presentes y correctamente identificados en el repositorio (código fuente, scripts de despliegue, archivos de configuración, esquemas de BD y documentación técnica). Responde a la pregunta: *¿Están todos los artefactos físicos necesarios en el repositorio y coinciden con la documentación de entrega?*

DISTINCIÓN DE EXAMEN: VERIFICACIÓN VS. VALIDACIÓN

* **Verificación:** Evalúa si los productos de una fase determinada satisfacen las condiciones impuestas en el inicio de dicha fase. Responde a: *¿Estamos construyendo el producto correctamente? (Conformance to specification).*

* **Validación:** Evalúa si el software satisface las necesidades y expectativas reales del cliente en su entorno operativo. Responde a: *¿Estamos construyendo el producto correcto? (Fitness for purpose).*

Trazabilidad Bidireccional Pre-ERS y Post-ERS

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 22-23) | filminas de clase (02, diapositivas 33-36)

La **Trazabilidad (Rastreabilidad)** es la capacidad técnica de seguir la vida y evolución de un requerimiento en ambas direcciones a lo largo del ciclo de vida del software:

Trazabilidad Hacia Atrás (Backward Traceability)

Permite vincular cada elemento de diseño, código o prueba con el requerimiento de software (ERS) que le dio origen, respondiendo a la pregunta: *¿Por qué existe este módulo de código o para qué se diseñó?*

Trazabilidad Hacia Adelante (Forward Traceability)

Permite mapear cada requerimiento hacia los módulos de diseño, clases de código y casos de prueba que lo implementan, respondiendo a: *¿Qué impacto en el sistema tendrá modificar este requerimiento específico?*

Trazabilidad Pre-ERS vs. Post-ERS

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (página 23) | filminas de clase (02, diapositiva 35)

- **Rastreabilidad Pre-ERS:** Registra los vínculos desde las necesidades iniciales expresadas por los usuarios y clientes en las minutas de relevamiento hasta su formalización como requerimientos concretos en la ERS.
- **Rastreabilidad Post-ERS:** Vincula los requerimientos aprobados de la ERS con los artefactos técnicos descendentes: modelo de arquitectura, diagramas de diseño, clases y métodos en el código fuente, y los casos de prueba asociados.

Gestión de Repositorios, Permisos y Buenas Prácticas de SCM

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 23-24) | filminas de clase (02, diapositivas 37-40)

La cátedra establece una serie de reglas operativas inmutables para el trabajo en repositorios:

1. **El Número de Versión NUNCA debe formar parte del nombre del archivo del IC:** En las herramientas modernas de versionado (como Git o SVN), el sistema gestiona internamente el historial de cambios y las etiquetas. Nombrar archivos como `Controlador_v2_final_corregido.java` es una aberración metodológica que destruye el historial unificado. El archivo se llama `Controlador.java` y su versión se consulta en el log del repositorio.
2. **El Plan de Gestión de Configuración (SCMP) es el PRIMER IC que debe entregarse:** Antes de iniciar el desarrollo técnico, el equipo debe acordar formalmente cómo se estructurará el repositorio, qué reglas de nombrado se aplicarán, qué políticas de branching se adoptarán y qué permisos de acceso tendrá cada integrante.
3. **Estructura y Permisos del Repositorio:** Se deben restringir los permisos de escritura directa en las ramas maestras de producción (`main` o `master`). Solo se admiten cambios mediante solicitudes de integración auditadas (*Pull Requests* o *Merge Requests*).

SCM en Entornos Ágiles y Cultura DevOps

Fuente: apunte de clase: ISW_TEORICO UNIDADES 1 Y 3 (páginas 21-22, 24) | filminas de clase (02, diapositivas 41-44)

La adopción de la filosofía ágil y la cultura DevOps no elimina la disciplina de SCM; la **transforma mediante la automatización radical**:

¿Qué pasa con el CCB en Agile?

El comité burocrático tradicional es reemplazado por la **Revisión de Código por Pares (Peer Code Reviews) a través de Pull Requests (PRs)**. Antes de fusionar un cambio a la rama principal, al menos un desarrollador del equipo debe inspeccionar el código, verificar que no vulnere la arquitectura y corroborar que las pruebas unitarias automatizadas pasen exitosamente.

¿Qué Ítems de Configuración tenemos?

En DevOps, **todo lo necesario para construir, probar y desplegar el sistema se convierte en código versionable (Everything as Code)**: código fuente, pruebas automatizadas, infraestructura como código (Terraform, CloudFormation), configuraciones de contenedores (Dockerfiles, Kubernetes manifests) y documentación viva (*Lean Documentation* en Markdown o Swagger/OpenAPI).

¿Qué ocurre con las Auditorías de SCM?

Se delegan por completo a los **Pipelines de Integración Continua y Despliegue Continuo (CI/CD)**. En cada commit, el pipeline compila el software (PCA automática), ejecuta la batería completa de pruebas unitarias, de integración y de seguridad (FCA automática) y genera reportes de cobertura de código sin intervención humana manual.

¿Qué ocurre con los Informes de Estado?

Los antiguos informes manuales son reemplazados por **Dashboards de Telemetría en Tiempo Real**: tableros Kanban/Scrum interactivos (Jira, GitHub Projects), registros inmutables de commits en Git, changelogs automáticos generados a partir de tags y herramientas de monitoreo en producción (Prometheus, Grafana).